

Bölüm 1 Sonlu Otakimeler

1.1. Sonlu Otakimeler Modeli

Kesitli gire ve gikileri olan matematiksel bir modeldir. Metin durumluysalar, deuyelerinin baki kumeler ve senkron ardissil devreler, sonlu otakimeler modeli ile gusekkeltilmektedir. baki bilgilerin.

iki sekilde siniflanabilir.

- Sonlu durumlu tanigici (finite state recognizer)
- Gireli devreler otakimeler.

1.1.1. Deterministik Sonlu Otakimeler Modeli (DFA)

DFA bir baki olarak tanimlanir.

$$DFA = \langle Q, \Sigma, \delta, q_0, F \rangle$$

- Q : Sonlu sayida durum iseren durumlar kumesi.
- Σ : Sonlu sayida gire sinisi iseren gire alfabeli.
- δ : Durum gecir islevi (state transition function) $(Q \times \Sigma) \rightarrow Q$ ya da bir eyleme.
- q_0 : Baslangic durumu ($q_0 \in Q$).
- F : Uci durumlar kumesi $F \subseteq Q$

q_i durumunda iken a geldigide q_j ye durumu gir.

$$\delta(q_i, a) = q_j \quad q_i \in Q, a \in \Sigma \Rightarrow q_j \in Q$$

DFA'da her an, matinein bulunduigi durum kisi olarak bittir.

Örnek

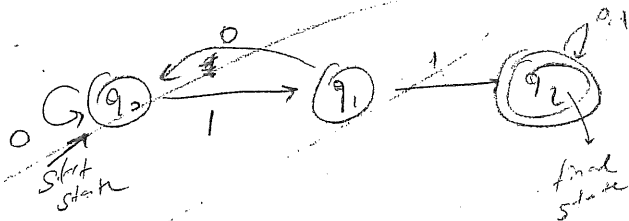
$$M_{1.1} = \langle Q, \Sigma, \delta, q_0, F \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_2\}$$

$$q_0 = \{q_0\}$$



- δ : $\delta(q_0, 0) = q_0$ Gire islevi mat. olarak baktiginiz her
- $\delta(q_0, 1) = q_1$ $\rightarrow$ ve nuni oldugundan durum gecir digre
- $\delta(q_1, 0) = q_0$ kullanilir.
- $\delta(q_1, 1) = q_2$
- $\delta(q_2, 0) = q_2$
- $\delta(q_2, 1) = q_2$

12 cm
2.5 cm

DFA'nın tanıdığı diziye tanıma.

DFA girisine her seferinde bir simge uygulanır. Başlangıçta q_0 durumunda bulunan DFA belirli bir gireye göre bir sonraki durumu belirler. Gire değeri w (string) olsun.

$$\delta(q_0, w) = q_f$$

$$q_0 \rightarrow w \rightarrow q_f$$

Eğer q_f q_f durumu DFA w dizisini tanıtır.

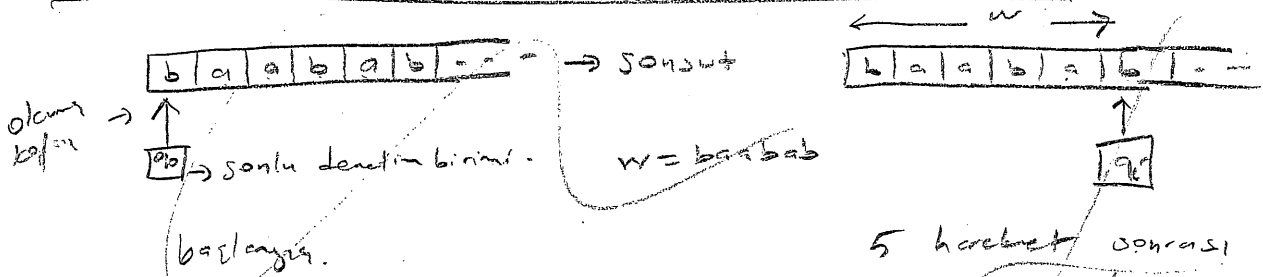
$$T(M) = \{ w \mid \delta(q_0, w) = q_f \in F \}$$

Bu durumda M_{11} DFA'si 0110 dizisini tanıtır.
0101 " tanıtır.

$$T(M_{11}) = \{ 1, 011, 110, 0110, 01011, 101011, \dots \}$$

M_{11} sonlu string tanıtır.
Sözlemler olarak ifade edilirse $T(M_{11})$ $\{0,1\}$ alfabesinde içinde 11 alt dizisi bulunan diziler kümesidir.

DFA Soyut Modelinin Seriler Makine Modeli.



- 5 hareket sonrası q_f q_f durumu w 'yi tanıtır.
1. Serilerin string okunması.
 2. Okuma kafası sağdaki hücreye geçer.
 3. SDB sonraki duruma geçer.

- Elementer:
1. Hücreler girilen stringden evir.
 2. Sonlu denetim birimi.
 3. Okuma kafası → soldan sağa doğru okur.

1.1.2. NFA Modeli

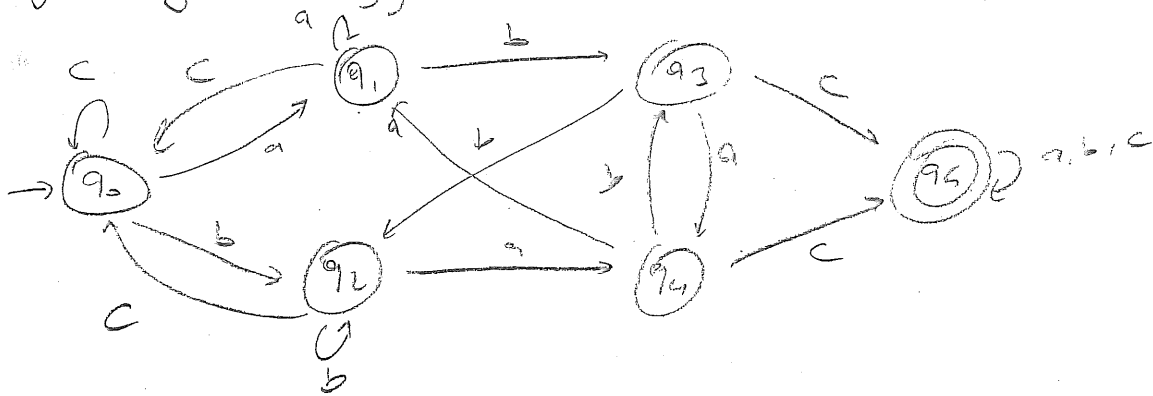
Sözler olarak verilen

Belirli bir diziye karşılık gelen tanıyan DFA'nın geçici diyagramını çizmek zor olabilir.

örnek // $\{a, b, c\}$ göre alfabe içinde, içinde abc veya bac alt dizilerinden en az bir en az bir kez bulunan diziye karşılık tanıyan DFA'ya çizim.

$$T(M_{1,2}) = \{ \underline{abc}, \underline{bac}, e \underline{b} a \underline{b} c b, b c c a \underline{a} b c b a c, \dots \}$$

Sözler tanımları kullanarak geçici diyagramı elde eden bir yöntem
Ayrıca geçici diyagramı oluşturmak mümkündür.



modelinin
DFA kullanımı gerektiren bazı NFA (Non-deterministik FA) geliştirilmiştir.

NFA bir besleyici olarak tanımlanır. (DFA gibi yet)

DFA ile farklı $Q \times \Sigma \rightarrow Q$ nun alt kümesine tanımlı olmazdır.

örnek 02-NFA text notes

$$M_{1,3} = \langle Q, \Sigma, \delta, q_0, F \rangle$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_3\}$$

$$\delta: \delta(q_0, 0) = \{q_0, q_1\}$$

$$\delta(q_0, 1) = \{q_0, q_2\}$$

$$\delta(q_1, 0) = \{q_3\}$$

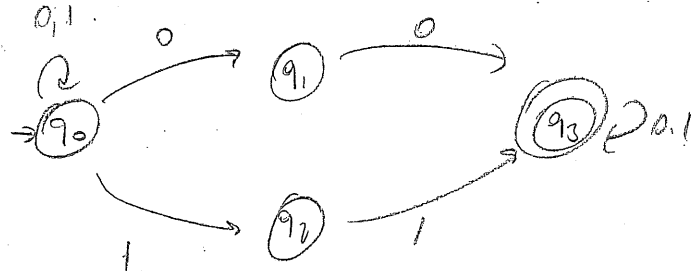
$$\delta(q_1, 1) = \emptyset$$

$$\delta(q_2, 0) = \emptyset$$

$$\delta(q_2, 1) = \{q_3\}$$

$$\delta(q_3, 0) = \{q_3\}$$

$$\delta(q_3, 1) = \{q_3\}$$



$w = 000$ uygulanırsa

bu NFA q_0, q_1 veya q_3 durumlarından herhangi birinde olabilir.

M1.3 kein ganz string
 $W = 10001$ verbleibende

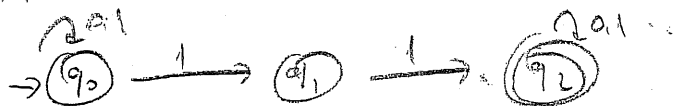
$q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{0} q_3 \xrightarrow{0} q_3 \xrightarrow{1} q_3$ } hier ist gut da q_3
 $q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{0} q_1 \xrightarrow{0} q_3 \xrightarrow{1} q_3$ } us durcheinander verbleibende
 $W = 10001$ M1.3 für fertig.

$W = 010$ kein

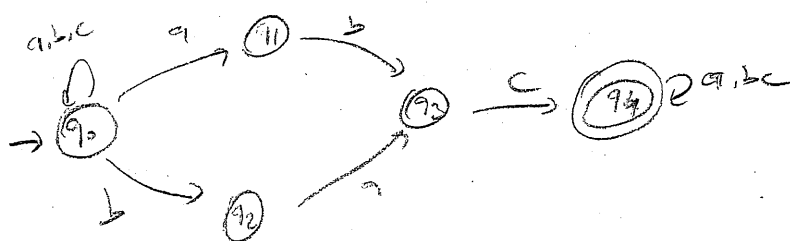
$q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0$ x q_3 e gut gut
 $q_0 \xrightarrow{0} q_1 \xrightarrow{1} ?$ x " "
 $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1$ x "
 $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_2 \xrightarrow{0} ?$ x "

M1.3 in "0,1" alphabet, sind 00 und 11 alphabeten
 en es bis, en at bis bis, beiden dig, le "en" or
 tonen, keine obigen gerichtete.

M1.1 ist NFA model.



M1.2 ist NFA model.



1.1.3 Lambda (λ) Geçirir

λ geçirir, hiçbir giriş sinyali uygulanmadan, makinenin kendine kendine durum değiştirir.

$$\delta(q_1, \lambda) = q_2$$

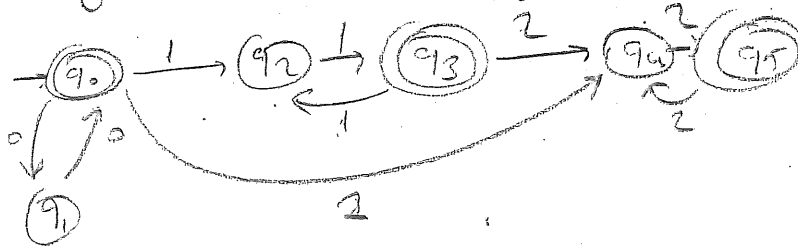
(q₁) $\xrightarrow{\lambda}$ (q₂) Makine kendiliğinden q₂ durumuna geçebilir.

λ geçirir tek yönlüdür.
Modelin çalışmasını arttıracak diğer bazı girişlerdir.
q₁ durumundaki makine aynı anda q₂ durumundadır.
fakat q₂ " " " " q₁ durumunda değildir.

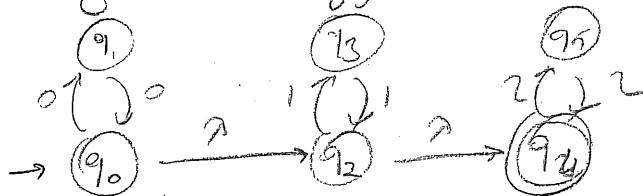
Örnek q₁ ve q₂ ye λ geçirir varsa
• q₁ başlangıç durumu ise q₂ de başl. durumudur.
• q₂ us. durum ise q₁ de us. durumudur.

$$\delta T(M_{1,4}) = \{0^* 1^* 2^* \mid n \geq 0, m \geq 0, k \geq 0\}$$

• λ geçirir ~~çalıştır~~ diyagram



• λ geçirir diyagram



λ geçirir diyagramı gizli deha katedir.

$\rightarrow$ geordnete erdige ~~geordnete~~ ^{gesie digt} ~~buknassi~~ ^{bir} $\rightarrow$ gesiezt
durchnr { Mithly $\rightarrow$ gesiezte dnh digt vord.
 q_1 vs q_2 arande q_1 der q_2 ye $\rightarrow$ gesiezt vorse

* q_2 durumundan b eylemleri her durum geçişine
 $\delta(q_2, a) = q_2$ karşılık, q_1 durumundan b eylemleri ve
aynı giriş stringi ile a aynı duruma ulaşan bir durum geçişi
* $\delta(q_1, a) = q_2$ eklenmiştir

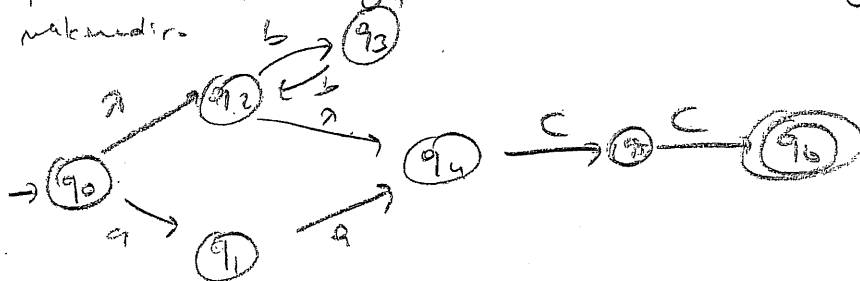
*2 Eier q₁ dur ball- durum wie q₂ d bei- dur mit passiv
 *3 Eier q₂ " wq durum wie q₁ d wq dur
 yppidelt +

sonra A geçire siliyorum

~~55~~ 49 durumda 299 dga jama.

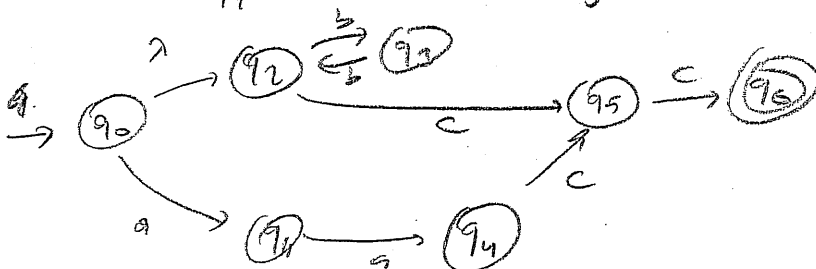
$$6/1: T(M_1, S) = \{cc, aacc, bbcc, bbbcc, bbbbbbcc, \dots\}$$

Mr. "Sifir ya da iki tane a ile ya da gift sayıları b ile ile barlayıp cc ile biten sayılar kim ^{istenen} istenir.
nakmedir. b → (93)



q₂ is q_n across n

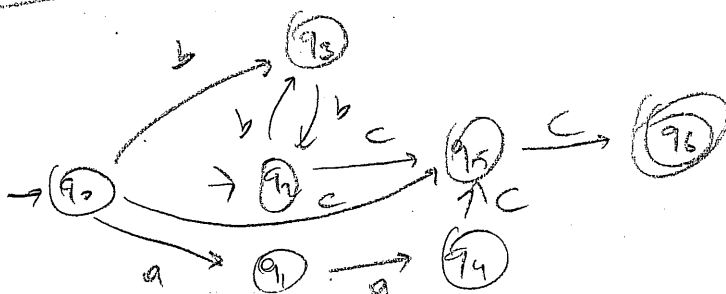
- $\delta(q_1, c) = q_5$ için $\delta(q_2, c) = q_5$ eklenir.
 • q_2 başlangıç durumu
 • q_5 son durum



90 Hc 92

$\text{OS}(q_2, c) = 95$ *ok*
 $\delta(q_0, c) = 95$ *abgleich*
 ② $\delta(q_2, b) = 92$ *ok*
 $\delta(q_0, b) = 92$ *abgleich*

• 7 bas. m de bas.
okur.



2. geometrisches Diagramm
elide wählen

114. DFA ve NFA Dönüştürme.

Her DFA aynı zamanda bir NFA'dır. Fakat her NFA bir DFA'ya dönüştürülebilir. Eğer NFA tarafından tanımlanmış bir dil için bir DFA varsa, modelin doğru olduğunu söyleyebiliriz.

Örnek: $M = \langle Q, \Sigma, \delta, q_0, F \rangle$

$$Q = \{A, B, C\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = A$$

$$F = \{C\}$$

$$\delta: \delta(A, 0) = \{A\}$$

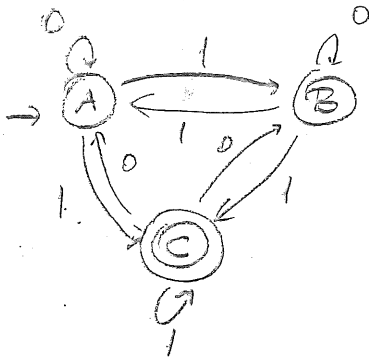
$$\delta(A, 1) = \{B, C\}$$

$$\delta(B, 0) = \{B\}$$

$$\delta(B, 1) = \{A, C\}$$

$$\delta(C, 0) = \{A, B\}$$

$$\delta(C, 1) = \{C\}$$



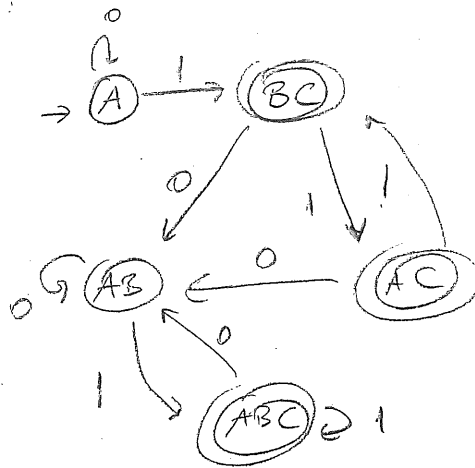
	0	1
A	A	BC
B	B	AC
C	AB	C

Durum Geçiş Tablosu

Durum geçiş diyagramı

Her DFA bulunurken bağımlı durumların ardışık durumlarına göre sıralıya göre sıralanır.

	0	1
A	A	BC
BC	AB	AC
AB	AB	ABC
AC	AB	BC
ABC	AB	ABC



Bağımlı durumların ardışık

- A: q_0
- BC: q_1
- AB: q_2
- AC: q_3
- ABC: q_4

genişletilmiş diyagram olarak gösterilebilir.

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$F = \{q_1, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$\delta: \delta(q_0, 0) = q_0$$

$$\delta(q_0, 1) = q_1$$

gibi.

1.1.5. İlk ysklk DFA Modeli

Temel DFA modeli tek ysklk ve okuma kafesi olarak soluma solden sağına dogru hareket eder.

2DFA modelinde de okuma kafesi gire singenir de duleten sonra bir solden veya bir sağına ~~hakkat~~ hareket eder.

2DFA bir besir olarak tanımlanır.

(DFA den tek farkı) $\delta: (Q \times \Sigma) \rightarrow Q \times \{L, R\}$ ye bir eslenir. (geçir isleminin tanımlanması)

Tüm durumlar uq durumdur ($F=Q$) ~~sağına~~ ~~geçir~~ ~~sağına~~ ~~geçir~~.

$$M_{1.7} = \langle Q, \Sigma, \delta, q_0, F \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_0, q_1, q_2\}$$

$$\delta: \delta(q_0, 0) = (q_0, R)$$

$$\delta(q_0, 1) = (q_1, R)$$

$$\delta(q_1, 0) = (q_1, R)$$

$$\delta(q_1, 1) = (q_2, L)$$

$$\delta(q_2, 0) = (q_0, R)$$

$$\delta(q_2, 1) = (q_2, L)$$

	0	1
$\rightarrow q_0$	(q_0, R)	(q_1, R)
q_1	(q_1, R)	(q_2, L)
q_2	(q_0, R)	(q_2, L)

2DFA nın diziyi tanıma sarti sonlara sonunda hareket sonunda, tüm diziyi islenir.

$$\text{Anlık tanım (ID): } \underbrace{x_0 x_1 x_2 \dots x_i}_{\alpha} \underbrace{x_{i+1} x_{i+2} \dots x_n}_{\beta} \quad ID_2(\alpha, q_i, \beta)$$

okuma kafesi

α : okunan singeler

β : kalan okunacak "

q_i : bulunduq duruma.

$ID_1 \vdash ID_2$: ID_1 den ID_2 ye tek okuma islemi sonunda gelinir.

$ID_1 \vdash^* ID_2$: " " " " " " " " " " " "

Buna göre anlık tanımlar arasındak isleme göre 2DFA nın tanıdığı diziler kümesi

$$T(M) = \{ w \mid (q_0, w) \vdash^* (w, q_i), q_i \in F \} \text{ dir.}$$

Başlangıç anlık tanımlar.

ID_0 'da α gektir.

son anlık tanımlar.

β string bitliğinden β gektir.

2/1 M_1 matnesi tanıfından $w_1 = 1000101$ ve $w_2 = 100110$ stringlerinin tanımlı formadığını inceleyelim.

a) * $w_1 = 1000101$ için

$(q_0, 1000101) \vdash (1, q_1, 000101) \vdash (10, q_1, 000101) \vdash (100, q_1, 0101) \vdash$
 $(1000, q_1, 101) \vdash (100, q_2, 0101) \vdash (1000, q_0, 101) \vdash$
 $(10001, q_1, 01) \vdash (100010, q_1, 1) \vdash (10001, q_2, 01) \vdash$
 $(100010, q_0, 1) \vdash (100010, q_1) \parallel \beta$ bitişinden M_1 bu stringi tanımlıdır.

* $w_1 = 1000101$

1	0	0	0	1	0	1
---	---	---	---	---	---	---

$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1$

$q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_1$
 $q_2 \rightarrow q_0 \rightarrow q_1$

b) $w_2 = 100110$

$(q_0, 100110) \vdash (1, q_1, 00110) \vdash (10, q_1, 0110) \vdash (100, q_1, 110) \vdash$
 $(10, q_2, 0110) \vdash (100, q_0, 110) \vdash (1001, q_1, 10) \vdash (100, q_2, 110) \vdash$
 $(10, q_2, 0110) \vdash (100, q_0, 110) \vdash \dots$

dişin üstünden w_2 M_1 tanıfından tanımlıdır.

1	0	0	1	1	0
---	---	---	---	---	---

$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1$

$q_2 \rightarrow q_0 \rightarrow q_1$
 $q_2 \rightarrow q_2$
 $q_0 \rightarrow q_1$
 $q_2 \rightarrow q_2$

RDFA'ya denk bir DFA
 bulunabilir fakat bu dersin
 dışındadır.

Sonlu Özdevinirler

- Sonlu durumlu tanıyıcı
- Giriş üreten özdevinir

olarak ikiye ayrıldığına baktığımızda

Tanıyıcı model giriş simgelerinden ~~string~~ stringlerin bir kısmını tanıyan modeldir. Syntax analiz, parsing gibi ^{lingual} uygulamalarda kullanılmaktadır.

Giriş üreten Özdevinir modeli ise, girişine uygulanan bir stringe karşılık, giriş stringi üreten modeldir.

Tanıyıcı modelin ~~string~~ ~~bulunabilen~~ ~~girişine~~ ~~uygulan~~ string formu 1 formu 0 olarak tanımlanır. Sonlu durumlu tanıyıcılar da giriş üreten özdevinirlere dahil edilebilir.

Giriş üreten özdevinirler durum düzeyinde giriş üreten Moore makinesi ve durum geçiş düzeyinde giriş üreten Mealy makinesi olarak ikiye ayrılmaktadır.

Moore Makinesi : Moore makinesi editör olarak tanımlanır.

$$M = \langle Q, \Sigma, \Delta, \delta, q_0, \lambda \rangle$$

Q : Sonlu durumlar kümesi

Σ : Giriş alfabesi (sonlu küme)

Δ : Giriş alfabesi (" ")

δ : Durum geçiş işlevi $(Q \times \Sigma) \rightarrow Q$ ya

q_0 : Başlangıç durumu $q_0 \in Q$

λ : Giriş fonksiyonu Q 'den Δ ya $Q \rightarrow \Delta$

DFA modeli aslında bir Moore makinesidir.

$$\Delta = \{0, 1\} \quad \forall q_i \in F \Rightarrow \lambda(q_i) = 1 \quad \forall q_i \notin F \Rightarrow \lambda(q_i) = 0$$

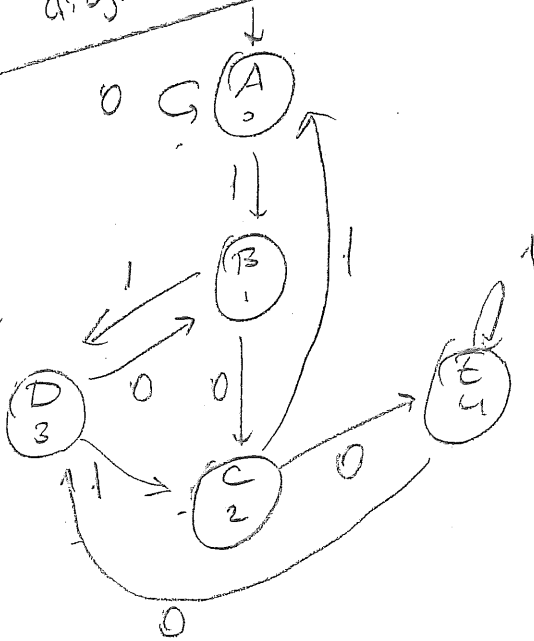
DFA modeli editör editör.

8700/1 M. 8



Aşağıya bakınız

M1.8 durum geçiş
diagramı



GİKİE Üreten Otomatlar.

FA
transisyonlar döngü transisyonları

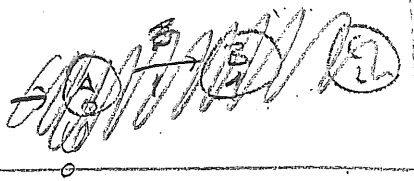
Moore Mealy durum geçiş düzeyinde

çıkış üretir

Moore

$M = \langle Q, Z, \Delta, \delta, \gamma, q_0 \rangle$

$\rightarrow$ çıkış fonks.
 $\rightarrow$ çıkış alfabesi



Girişi X göre çıkışına $Z = \text{mod}(X, 5)$ değeri üreten makine

$M_{1.8} = \langle Q, Z, \Delta, \delta, \gamma, q_0 \rangle$

çıkışlar belirlendi
01234 stabilite 5'inci olarak

$Q = \{A, B, C, D, E\}$ $Z = \{0, 1\}$ $\Delta = \{0, 1, 2, 3, 4\}$ $q_0 = A$

SD	0	1	out Z
A	A	B	0
B	C	D	1
C	E	A	2
D	B	C	3
E	D	E	4

I : 1 1 1 1 0
S : A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ C $\rightarrow$ A $\rightarrow$ A
O : 0 1 3 2 0 0

$\rightarrow$ B $\xrightarrow{0}$ C $\xrightarrow{0}$ E $\xrightarrow{0}$ D $\xrightarrow{0}$ B
1 2 4 3 1

976/5
(1)

Mealy

$$M = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$$

Moore ile tek fark λ çıkış referansidir.

M1.7.

Moore $Q \rightarrow \Delta$
Mealy $(Q \times \Sigma) \rightarrow \Delta$

$$M_{1.9} = \langle Q, \Sigma, \Delta, q_0, \delta, \lambda \rangle$$

$\Sigma = \{0, 1\}$ $\Delta = \{0, 1, 2\}$. olan ve doğru çıkış olan son iki giriş sinyallerinden biri değişimin bir öncekinden farklı oldu. bütün mealy makinesini tasarlayın. Σ

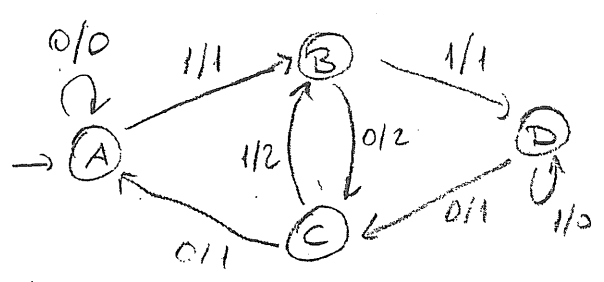
$$X: 00011010101111010011100001$$

$$Z: 01112222100122111011001$$

		Σ, Z	
		0	1
00	$\rightarrow A$	A, 0	B, 1
01	B	C, 2	D, 1
10	C	A, 1	B, 2
11	D	C, 1	D, 0

Çıkış ne stabil?
00, 01, 10, 11 durumuna göre geçişler. 6-7 altındır.

00 A } Hangi durumda olur
01 B } olur
10 C }
11 D }
00 $\rightarrow A$
01 $\rightarrow B$
10 $\rightarrow C$
11 $\rightarrow D$ 'ye gider.



Moore Esdgeri Mealy

$$q_0 \xrightarrow{x_1} q_1 \xrightarrow{x_2} q_2 \dots$$

$z_1 \quad z_2$

$$q_0 \xrightarrow{x_1/z_1} q_1 \xrightarrow{x_2/z_2} q_2 \dots$$

Moore

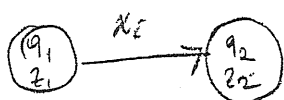
Mealy

M_1 moore ise esdgeri M_2 mealy

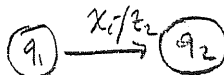
$$M_1 = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$$

$$M_2 = \langle Q, \Sigma, \Delta, \delta, \lambda', q_0 \rangle$$

$$\lambda'(q, x) = \lambda(\delta(q, x))$$



$\Rightarrow$



Al. r : hatirlayim. $t = m + (x, \delta)$

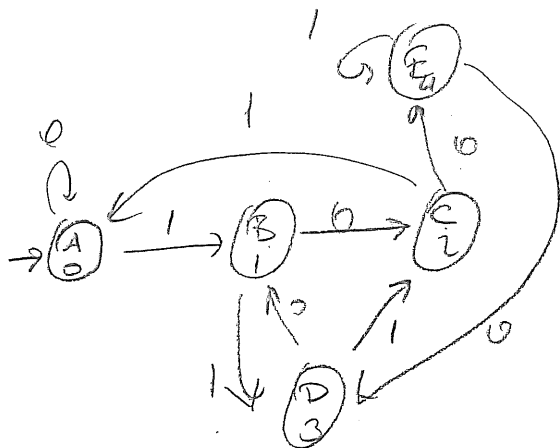
SD	0	1	out
A	A, 0	B, 1	0
B	C, 2	D, 3	1
C	E, 4	A, 0	2
D	B, 1	C, 2	3
E	D, 3	E, 4	4

Moore

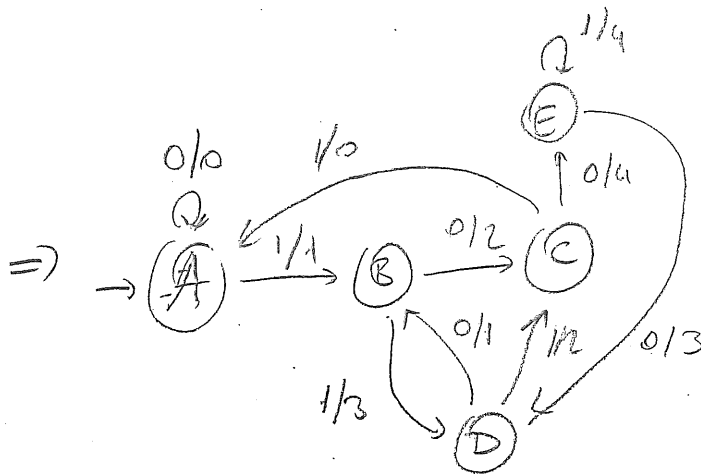
Mealy

$\Rightarrow$

SD	x=0	x=1
A	A, 0	B, 1
B	C, 2	D, 3
C	E, 4	A, 0
D	B, 1	C, 2
E	D, 3	E, 4



Moore



Mealy

Mealy Eşdeğer Moore

Durumlar, başlangıç durumu, geçiş ve çıkış fonksiyonları

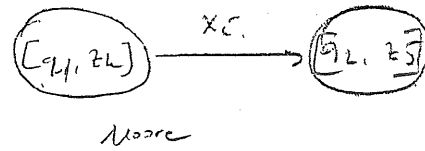
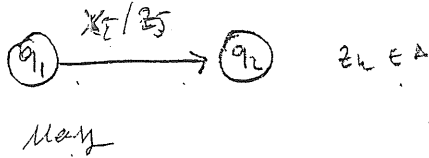
(12)

$$M_2 = \langle Q, Z, A, \delta, \lambda, q_0 \rangle \Rightarrow M_1 = \langle Q', Z, A, \delta', \lambda', q_0' \rangle$$

$$Q' = Q \times A \quad q_0' = [q_0, z_0] \quad (z_0 \text{ random çıkış sign})$$

$$\delta'([q_i, z_k], x_j) = [\delta(q_i, x_j), \lambda(q_i, x_j)]$$

$$\lambda'([q_i, z_k]) = z_k$$



$$M_{1,10} = \langle Q, Z, A, \delta, \lambda, q_0 \rangle$$

$$Q = \{A, B, C\} \quad Z = \{0, 1\} \quad A = \{0, 1\} \quad q_0 = A$$

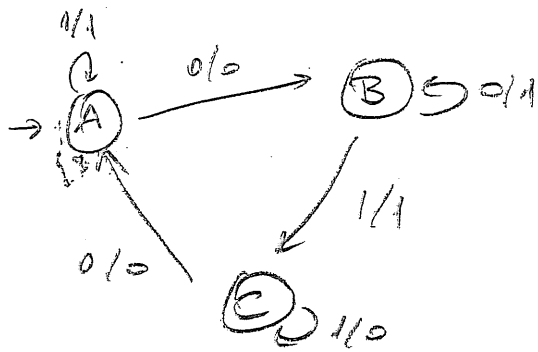
$\delta(A, 0) = B$	$\lambda(A, 0) = 0$
$\delta(A, 1) = A$	$\lambda(A, 1) = 1$
$\delta(B, 0) = B$	$\lambda(B, 0) = 1$
$\delta(B, 1) = C$	$\lambda(B, 1) = 1$
$\delta(C, 0) = A$	$\lambda(C, 0) = 0$
$\delta(C, 1) = C$	$\lambda(C, 1) = 0$

Verilen Mealy

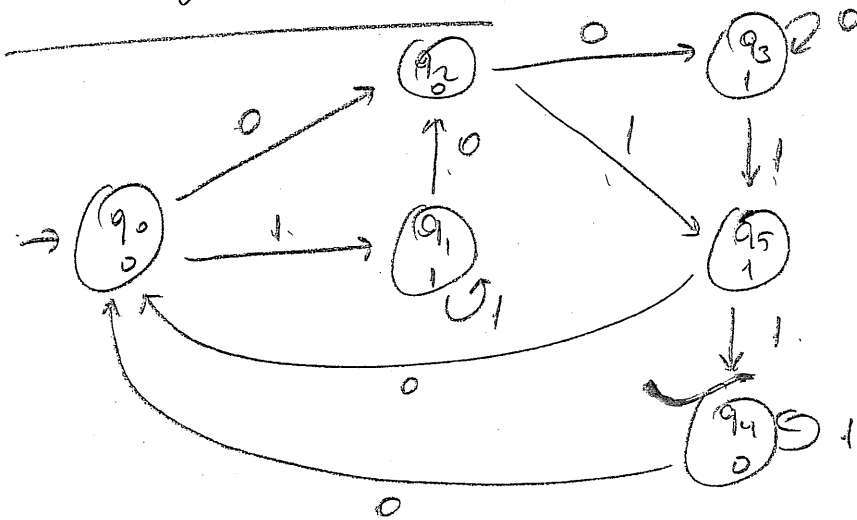
mek-e eşdeğer Moore makinesi

$$Q' = \{[A, 0], [A, 1], [B, 0], [B, 1], [C, 0], [C, 1]\} \quad q_0' = [A, 0]$$

$\delta'([A, 0], 0) = [B, 0]$	$\lambda'([A, 0]) = 0$	$[A, 0] \Rightarrow q_0$
$\delta'([A, 0], 1) = [A, 1]$	$\lambda'([A, 1]) = 1$	$[A, 1] \Rightarrow q_1$
$\delta'([A, 1], 0) = [B, 0]$	$\lambda'([B, 0]) = 0$	$[B, 0] \Rightarrow q_2$
$\delta'([A, 1], 1) = [A, 1]$	$\lambda'([B, 1]) = 1$	$[B, 1] \Rightarrow q_3$
$\delta'([B, 0], 0) = [B, 1]$	$\lambda'([C, 0]) = 0$	$[C, 0] \Rightarrow q_4$
$\delta'([B, 0], 1) = [C, 1]$	$\lambda'([C, 1]) = 1$	$[C, 1] \Rightarrow q_5$
$\delta'([B, 1], 0) = [B, 1]$		
$\delta'([B, 1], 1) = [C, 1]$		
$\delta'([C, 0], 0) = [A, 0]$	$\delta'([C, 1], 0) = [A, 0]$	
$\delta'([C, 0], 1) = [C, 0]$	$\delta'([C, 1], 1) = [C, 0]$	



Mealy machine



Edge Node Machine

#FA indirgenir

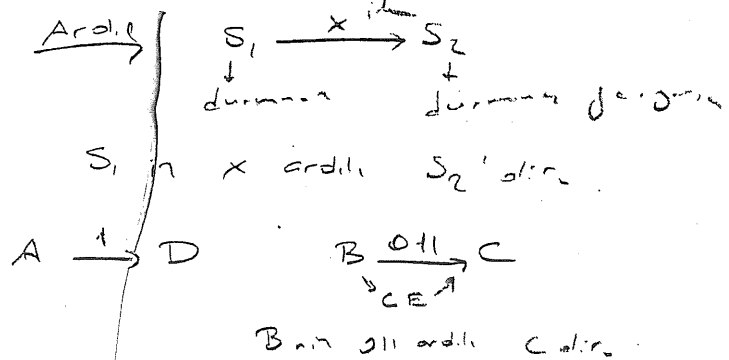
Bu sonlu FA'ya denli durum sayı en az olan FA'nı bulunmalıdır.

* Ardıl, Öncel, Denk ve Ayrılabılır Durum

M1.11 mesaly

SD, τ

SD	$x=0$	$x=1$
→ A	A, 0	D, 1
B	C, 0	E, 1
C	G, 0	E, 1
D	G, 0	F, 1
E	E, 1	C, 0
F	B, 0	D, 1
G	B, 0	E, 1



Öncel

$S_1, S_2, \dots, S_i \xrightarrow{x} S_j$ durumuna geçiliyorsa S_j 'nin x-önceli $\{S_1, S_2, \dots, S_i\}$ 'dir.

M1.11 için. B'nin 0 önceli FG, 001 önceli AEF'dir.

M1.11 öncel. tablosu.

SD	$x=0$	$x=1$
A	A	-
B	FG	-
C	B	E
D	-	AF
E	E	BCG
F	-	D
G	CD	-

Denk Durum

Maks. denk durum bulunmalı.

B ve D 1 denli.

B $\xrightarrow{x=0, z=0}$ C D $\xrightarrow{0, 0}$ G

B $\xrightarrow{1, 1}$ E D $\xrightarrow{1, 1}$ F

Makine S_1 ve S_2 durumlarında iken, hangi giriş sinyesi

uygulanırsa uygulanır

makine hep aynı çıkış sinyesini üretiyorsa bu durumlara

n-denli durum denir.

n+1 ayrı edilebilir.

A ve F 2 denli.

A $\xrightarrow{01, 01}$ D F $\xrightarrow{01, 01}$ E

A ve F 3 ayrılabılır.

A $\xrightarrow{010, 010}$ G F $\xrightarrow{010, 011}$ E

Ayrılabılır durum

(n-1) denli ise n ayrılabılır

A $\xrightarrow{0}$
A $\xrightarrow{1}$

F $\xrightarrow{0}$
F $\xrightarrow{1}$

A $\xrightarrow{0}$
A $\xrightarrow{01}$
A $\xrightarrow{10}$
A $\xrightarrow{11}$

F $\xrightarrow{0}$
F $\xrightarrow{01}$
F $\xrightarrow{10}$
F $\xrightarrow{11}$

Makine indirgenmesi

M_{min} veya M^*

Denklik Bölünmesi $M_{1..n}$ için

P_L indirgenmiş
↑ modelin durumu
1erini verile
olana kadar türettilir.

$P_0 = (ABCDEFGG)$ $P_1 = (ABCDFFG)(E)$ $P_L = \dots$ $P_{L+1} = P_L$

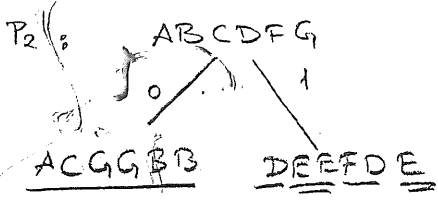
- a) S_1 ve S_2 M'de olduğu
- b) x için S_1 ve S_2 x'inde olduğu

Mealy'nin indirgenmesi

$P_0 = (ABCDEFGG)$

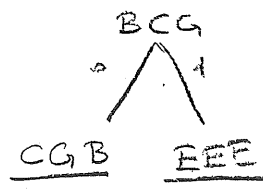
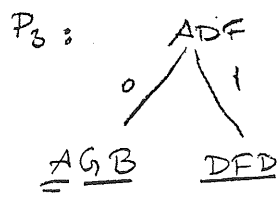
↳ Mealy makinesinin bütün durumları
denklikte hiçbir işi yapmayan
siteler olan Mealy makinesinin durumları
ayrıştırılabilir.

$P_1 = (ABCDFFG)(E)$



$P_2 = (ADF)(BCG)(E)$

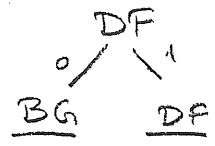
kondisyonlar
ADF ve BCG ile çalışır



$P_3 = (A)(DF)(BCG)(E)$

$P_4 = BCG$

asimptotik
incelikler
gözetilmez



$P_4 = P_3 = (A)(DF)(BCG)(E)$
tamamlandı.

A	0	S_0
DF	.	S_1
BCG	"	S_2
E	"	S_3

SD	SD, τ	
	$x=0$	$x=1$
S_0	$S_0, 0$	$S_1, 1$
S_1	$S_2, 0$	$S_1, 1$
S_2	$S_2, 0$	$S_3, 1$
S_3	$S_3, 1$	$S_2, 0$

P1 denklik bölünmesi için durum tablosu incelenir.
 $M_{1..n}$ de E hariç tüm durumlar 0 girilince 0
gikisi 1 girilince 1 çıkıyor. Birleştirilir.
E dışındaki tüm durumlar 1 çıkıyor.

(15)

Moore indirgenmesi

Mealy * de

Tek fark P_0 başlangıcı.

M112

SD	SD		z
	x=0	x=1	
→ A	C	B	0
B	B	D	1
C	A	H	2
D	E	G	1
E	C	D	2
F	C	H	2
G	G	H	1
H	F	B	1

$$P_0 = (A)(BDGH)(CEF)$$

$$P_1: BDGH$$

$$\begin{array}{c} 0 \quad 1 \\ \diagdown \quad \diagup \\ \underline{BEGF} \quad \underline{DGH B} \end{array}$$

$$CEF$$

$$\begin{array}{c} 0 \quad 1 \\ \diagdown \quad \diagup \\ \underline{ACC} \quad \underline{H D H} \end{array}$$

$$P_1 = (A)(BG)(DH)(C)(EF)$$

$$\begin{array}{ccc} \begin{array}{c} BG \\ 0 \quad 1 \\ \diagdown \quad \diagup \\ \underline{BG} \quad \underline{DH} \end{array} & \begin{array}{c} DH \\ 0 \quad 1 \\ \diagdown \quad \diagup \\ \underline{EF} \quad \underline{GB} \end{array} & \begin{array}{c} EF \\ 0 \quad 1 \\ \diagdown \quad \diagup \\ \underline{CC} \quad \underline{DH} \end{array} \end{array}$$

$$P_2 = P_1 = (A)(BG)(DH)(C)(EF)$$

$$S_0 \quad S_1 \quad S_2 \quad S_3 \quad S_4$$

SD	x=0	x=1	z
S_0	S_3	S_1	0
S_1	S_1	S_2	1
S_2	S_4	S_1	1
S_3	S_0	S_2	2
S_4	S_3	S_2	2

İndirgenmiş ille durum sayımı
8 → 5'e düşüyor.

DFA kisitli Moore makinesi.
Moore c.kisi {kabul, red} ile
eşlenen DFA model çıkarılır.

DFA indigum

SD	x=0	x=1
→ q ₀	q ₀	q ₁
q ₁	q ₂	q ₄
q ₂	q ₄	q ₇
q ₃	q ₆	q ₅
q ₄	q ₅	q ₃
(q ₅)	q ₅	q ₇
q ₆	q ₇	q ₂
(q ₇)	q ₇	q ₅

$$P_0 = (q_0, q_2, q_3, q_4, q_6) (q_5, q_7)$$

$$P_1: \begin{array}{c} q_0, q_1, q_2, q_3, q_4, q_6 \\ \swarrow \quad \searrow \\ 0 \quad 1 \end{array}$$

$$\underline{q_0, q_2, q_3, q_4, q_5, q_7} \quad \underline{q_1, q_4, q_7, q_5, q_3, q_2}$$

$$\begin{array}{c} q_5, q_7 \\ \swarrow \quad \searrow \\ 0 \quad 1 \end{array}$$

$$\underline{q_5, q_7} \quad \underline{q_7, q_5}$$

$$P_1 = (q_0, q_1) (q_2, q_3) (q_4, q_6) (q_5, q_7)$$

$$P_2: \begin{array}{c} q_0, q_1 \\ \swarrow \quad \searrow \\ 0 \quad 1 \end{array} \quad \begin{array}{c} q_1, q_3 \\ \swarrow \quad \searrow \\ 0 \quad 1 \end{array} \quad \begin{array}{c} q_4, q_6 \\ \swarrow \quad \searrow \\ 0 \quad 1 \end{array} \quad \begin{array}{c} q_5, q_7 \\ \swarrow \quad \searrow \\ 0 \quad 1 \end{array}$$

$$\underline{q_0, q_2} \quad \underline{q_1, q_4}$$

$$\underline{q_4, q_6} \quad \underline{q_7, q_5}$$

$$\underline{q_5, q_7} \quad \underline{q_3, q_2}$$

$$\underline{q_5, q_7} \quad \underline{q_7, q_5}$$

$$P_3 = P_2 = (q_0) (q_1) (q_2, q_3) (q_4, q_6) (q_5, q_7)$$

$$s_0 \quad s_1 \quad s_2 \quad s_3 \quad s_4$$

SD	x=0	x=1
→ s ₀	s ₀	s ₁
s ₁	s ₂	s ₃
s ₂	s ₃	s ₄
s ₃	s ₄	s ₂
(s ₄)	s ₄	s ₄

2.1. Düzgün Kümeler

Her sonlu özdüzen, given alfabesindeki simgelerden oluşan dizilerin bir kısmını tanımlar, bir kısmını tanımlar.

$$P_1 = \{1001, 0110\}$$

P_2 : III alt dizisinin içeren diziler kümesi.

(7)

P_3 : III ve son simgesi 1 olan en az 5 uzunluğundaki diziler kümesi.

$$P_4 = \{0^n 1^m \mid n \geq 1, m \geq 1\}$$

Verilen küme, tanıyan en az bir özdüzen varsa düzgün kümedir.

$$P_5 = \{0^n 1^n \mid n \geq 1\}$$

ve P_6 içinde eşit sayıda 0 ve 1 içeren diziler kümesi.

P_5 ve P_6 kümeleri sonlu olduğundan düzgün değildirler.

P_5 için:

n sayısı büyüdükçe sayıdan daha büyük olabilir. Ayrıca 0'ın eşit olduğu sayı da tutulmalıdır. Bu durumda P_5 sonlu olmaz.

2.2. Düzgün Deyimler

Her düzgün deyim, belirli bir alfabadeki simgelerden oluşturulan dizinin bir alt kümesidir tanımlar.

$\{a, b, c, \dots\}$ alfabesindeki düzgün deyimler:

1. Alfabadeki her simge bir düzgün deyimdir.

$$a = \{a\},$$

$$b = \{b\}, \dots$$

2. λ ve $\emptyset$ birer düzgün deyimdir.

$$\lambda = \{\lambda\}$$

$$\emptyset = \{\}$$

3. P ve Q düzgün deyim ise

$$P+Q = PQ$$

$P+Q$, PQ , P^* da düzgündür.

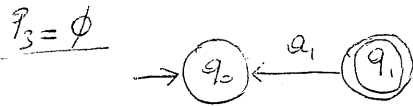
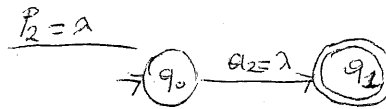
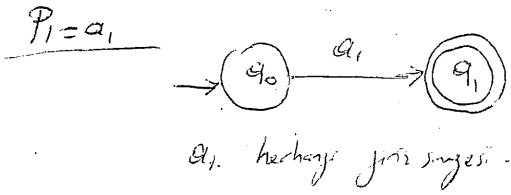
$$P.Q = \{PQ \mid P \in P, Q \in Q\}$$

$$P^* = \lambda + P + PP + PPP + PPPP + \dots$$

Örnekler

Düzen	Değer	Tanım kodu, Küm
$\emptyset$		$\{\}$
λ		$\{\lambda\}$
$00+11$		$\{00, 11\}$
$a(b+c)$		$\{ab, ac\}$
ab^*		$\{a, ab, abb, abbb, abbbb, \dots\}$
$a(bb+cc)d^*$		$\{abbb, abbbd, abbbdd, \dots, acc, accd, accdd, \dots\}$
a^*		$\{\lambda, a, aa, aaa, aaaa, \dots\}$
$(0+1)^*$		$\{\lambda, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, \dots\}$
$a(b+cd^*)^*a$		$\{aa, abaa, acaa, acda, acdda, abca, acddda, \dots\}$

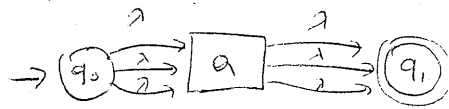
2.2.1. Düzen Değerlere Karşı Gelen Sonlu Ördemlerin Bulunması



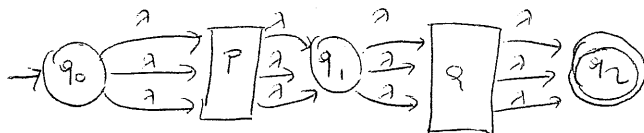
P yu tanıyan jecir cıterıgı.



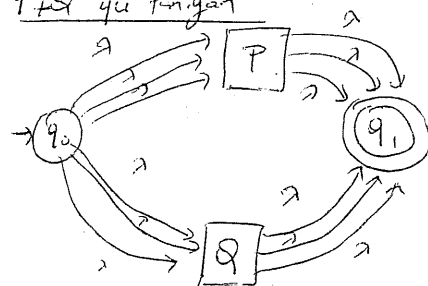
Q yu tanıyan jecir cıterıgı



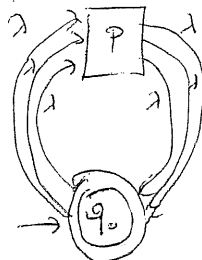
$P.Q$ yu tanıyan



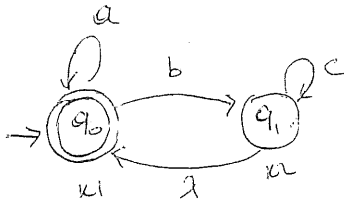
$P+Q$ yu tanıyan



P^* tanıyan



$(a + bc^*)^*$ birimsel yaklaşımla fazla stack ve λ gerektirir.
Sezgisel yaklaşıma göre çizim.



(8)

* Grup çizimlerinin sezgisel yaklaşımla oluşturulması sırasında dikkat edilecek önemli hususlar vardır:

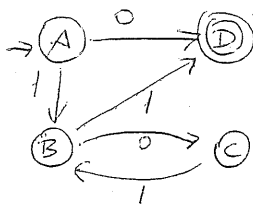
* 1. Eğer düğün içinde $\dots + ab$ ya da $(a^*b\dots)^*$ gibi bir örüntü varsa, üzerinde a döngüsü bulunan düğüme λ ile geçimlidir.

* 2. Eğer düğün içinde $\dots ab^* + \dots$ ya da $(ab^*)^*$ gibi bir örüntü varsa b döngüsü olan düğünden λ ile geçimlidir.

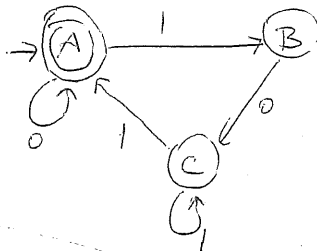
* 3. Eğer düğün içinde

$\dots a^*b^* \dots$ olan düğünden b döngüsü olan düğüme λ ile geçimlidir.

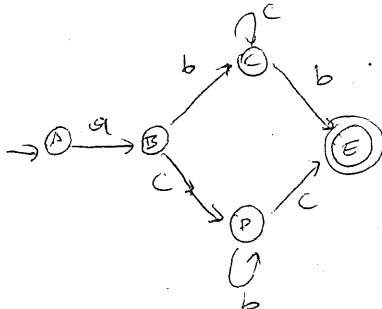
ör/ $P = 0 + 1(01)^*1$



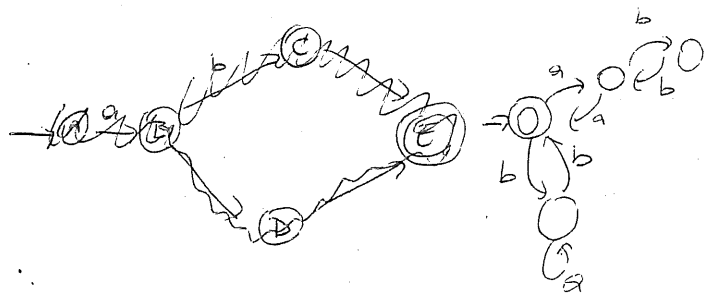
$P = (0 + 101^*1)^*$



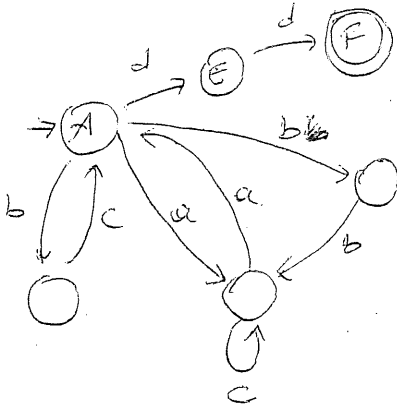
$P = a(bc^*b + cb^*c)$



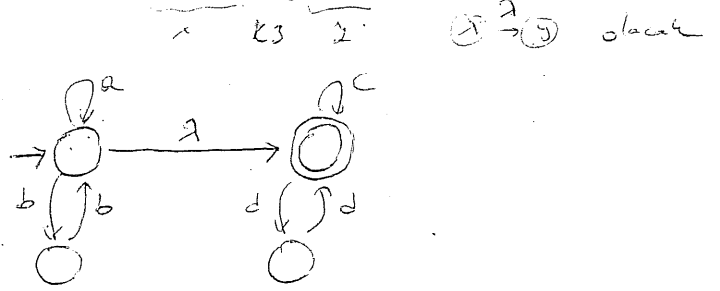
$P = (a(bb^*a + ba^*b))^*$



$$P = (bc + (a+bb)c^*a)^*dd$$



$$P = (a+bb)^*(c+dd)^*$$



2.2.2. Sonlu Özdevirilebilirlik Kuramının Düzgün Durum Olarak Bulunması

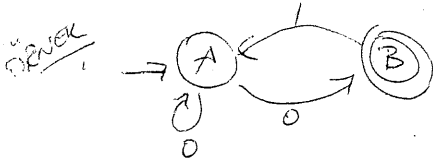
Teoremi: P, Q ve R aynı alfabe üzerinde tanımlanmış düzgün ifadelerse ve $P \stackrel{}{\sim} Q$ ise $P \stackrel{*}{\sim} R$ ise $R = Q + RP$ denkleminin tek çözümü $R = QP^*$ dir.

$$R = Q + RP \quad \text{denkleminin tek çözümü} \quad R = QP^* \text{ dir.}$$

Tek başlangıç durumu bulunan bir ifade girildiğinde her durum için aynı adı taşıyan bir düzgün ifade tanımlanır. A durumu için tanımlanan A değişkeni, başlangıç durumundan başlayıp A durumunda son bulan tüm diziye içeren bir kümeye karşılık gelir. Her A durumu için, sol tarafında bir değişken, sağ tarafında ise A durumunda son bulan her

$$\delta(B, a) = A \quad \text{geçerli bir Bar ifadesinin yer aldığı}$$

$A = \sum \text{Bar}$ biçiminde bir denklem kurulum Teoremi uygulanarak denklem sistemi çözülür ve her değişken için bir düzgün ifade elde edilir.



Makinenin A ve B durumu olduğu için iki değişkenli denklem kurulur.

$$A = a + Aa + Bb \quad (1)$$

$$B = Aa \quad (2)$$

Denklem (1)'de B'nin yerine denklem (2)'deki AO değeri konulduğunda

$$A = \lambda + AO + AO = \lambda + A(0+01) \quad (3)$$

(3). denklem verilen Teorem göre çözülmüş

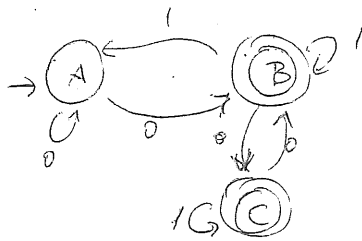
$$A = \lambda(0+01)^* = (0+01)^* \quad (4) \quad \text{elde edilir.}$$

(9)

Denklem 2 de A'nın yerine (4)'te elde edilen değeri koyarsak

$$B = AO = (0+01)^*0 \quad \text{Tek uçuş durum B olduğundan } T(u) = (0+01)^*0 \text{ olur}$$

örnek



Makinenin A, B, C gibi üç durumu olduğundan en düşükteki denklem kurulur.

$$A = \lambda + AO + BI \quad (1)$$

$$B = AO + BI + CO \quad (2)$$

$$C = BO + CI \quad (3)$$

Üçüncü denkleme Teorem 2.3 uygulanırsa.

$$C = BO1^* \quad (4) \quad \text{elde edilir. 2. denkleme C yerine } BO1^* \text{ konulursa}$$

denklem

$$B = AO + BI + BO1^*0 = AO + B(1+01^*0) \quad (5) \quad \text{Teorem yeniden kullanılarak çözülmüş}$$

$$B = AO(1+01^*0)^* \quad (6) \quad (6). 1. denkleme B yerine konursa.$$

$$A = \lambda + AO + AO(1+01^*0)^* = \lambda + A[0+0(1+01^*0)^*1] \quad (7) \quad \text{Teorem}$$

$$\text{yeniden çözülmüş } A = \lambda[0+0(1+01^*0)^*1]^* = [0+0(1+01^*0)^*1]^* \quad (8)$$

Denklem (6) da A yerine (8)'de bulunan değeri konulursa

$$B = [0+0(1+01^*0)^*1]^*0(1+01^*0)^* \quad (9) \quad \text{denk (4)'te B yerine 9'da bul.}$$

$$C = [0+0(1+01^*0)^*1]^*0(1+01^*0)^*01^*$$

$$T(A) = B + C = B + BO1^* = B(\lambda + 01^*)$$

$$T(A) = [0+0(1+01^*0)^*1]^*0(1+01^*0)^*(\lambda+01^*)$$

B2 Düzgün Kümeler ve Düzgün Deyimler.

Ör $\{0,1\}$ alfabesinde aşağıdaki FA tarafından tanımlanan kümeler tanımlanabilir.

$$P_1 = \{1001, 0110\}$$

$$P_2 = 111$$

ardışık 3 tane 1 olan

P_3 : ilk üyesi 1 olan en sol 5 üyeli.

$$P_4 = \{0^n 1^m \mid n \geq 1, m \geq 1\}$$

Sonlu setler tarafından tanımlanan kümeler düzgün kümeler denir. (regular sets)

Eğer küme tanıyan en az 1 FA varsa küme düzgündür.

Düzgün kümeler biçimsel olarak tanımlanmış için kullandıkları düzgün deyim denir.

Tanım: $\{a, b, c, \dots\}$ alfabesindeki düzgün deyimler.

1. Alfabedeki her a, b bir d.d.d.r.
 $a = \{a\}$, $b = \{b\}$, ...

2. λ ve $\emptyset$ d.d.d.r.
 $\lambda = \{\lambda\}$ $\emptyset = \{\}$

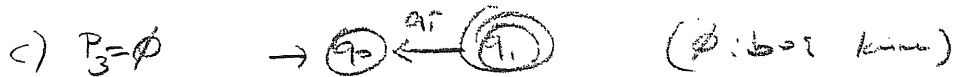
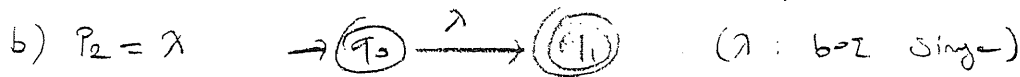
3. Eğer P ve Q d.d. ise

$P+Q$ | PQ | P^* da d.d. dir
 $= P \cup Q$ | ardışık | kopyalar

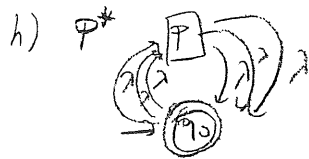
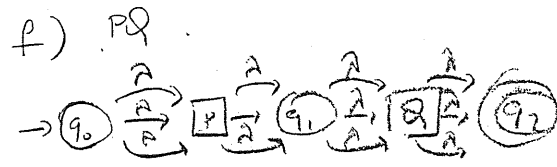
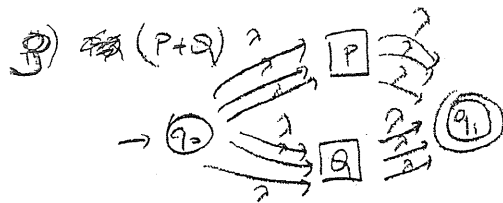
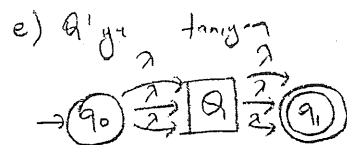
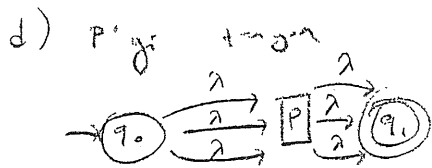
4. Bu da kurallara sonlu sayıda kopya ile oluşturulan deyimler d.d. dir.

D.P.	$\{a, b, c, d\}$ ve $\{0,1\}$ Tanımladığı Küme.	$a(b+cd^*)^*a$ $\{aa, abba, abbbba, aca, acada, abccdddba\}$ a^* $\{\lambda, a, aa, aaa, \dots\}$ $(0+1)^*$ $\{\lambda, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$
$\emptyset$	$\{\}$	
λ	$\{\lambda\}$	
$00+11$	$\{00, 11\}$	
$a(b+c)$	$\{ab, ac\}$	
ab^*	$\{a, ab, abb, abbb, abbbb, \dots\}$	
$a(b+cc)^*d$	$\{abb, acc, abbd, accd, ribbd, accdd, \dots\}$	

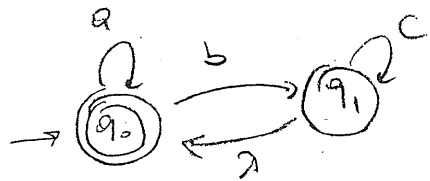
D. D'e Kari Gelen FA Lubunması.



~~P ve Q dıgı dıgı oluk uıerı~~



örnek: $P = (a+bc^*)^*$



$\rightarrow$ Sağsal yöntem

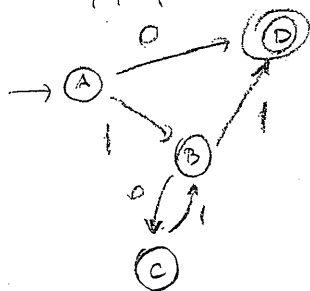
1. a^*b^* veya $(a^*b^*)^*$ varsa
a dıgı b dıgıne λ dıgıne

2. ab^* veya $(...ab^*)^*$ varsa
b dıgıne λ dıgıne

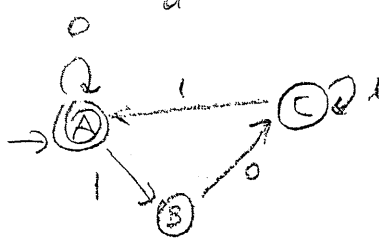
3. a^*b^* varsa
a dıgıne b dıgıne λ dıgıne

19

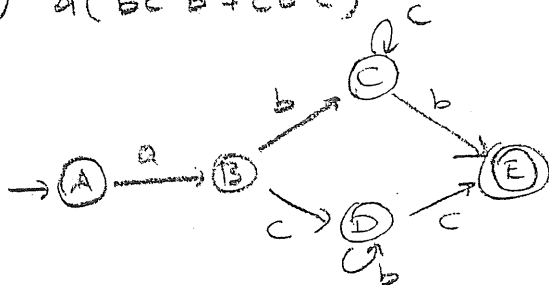
a) $P_1 = 0 + 1(01)^*1$
 $P+Q$



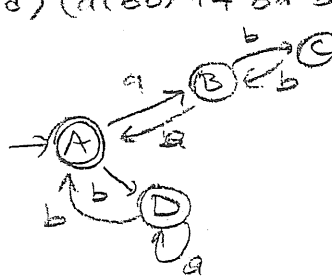
b) $(0 + 101^*1)^*$
 a^*



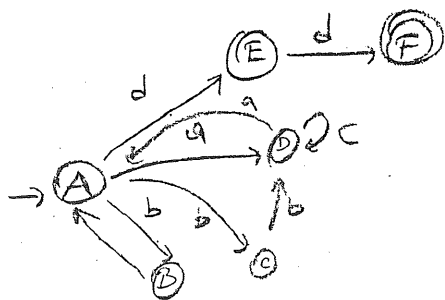
c) $a(bc^*b + cb^*c)$



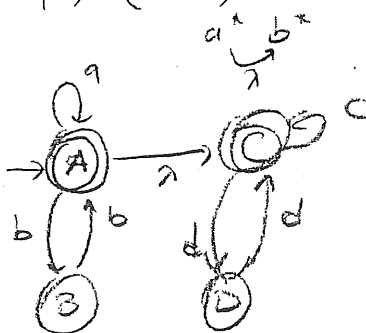
d) $(a(bb)^*a + ba^*b)^*$



e) $(bca + (a+bb)c^*a)^*dd$



f) $(a+bb)^*(c+dd)^*$



1	2	
0	0	
0	1	
1	0	
1	1	2.

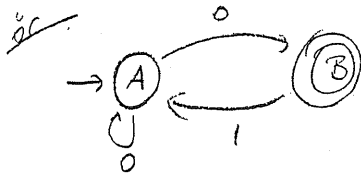
#FA Tanıdığı Kümelerin Düzgün Deyim Olarak Bulunması

(20)

* P, Q, R aynı alfabe üzerinde tanımlanmış dsl. ise ve P $\cap$ $\emptyset$ 'a karşılık gelmiyorsa $R = P + QP$ denli. tek çözüm $R = QP^*$

$$\delta(B, a_i) = A$$

$$A = ZB a_i$$



A ve B iki durumu old. denli. denli. kumeler

$$A = \lambda + A0 + B1 \quad (1)$$

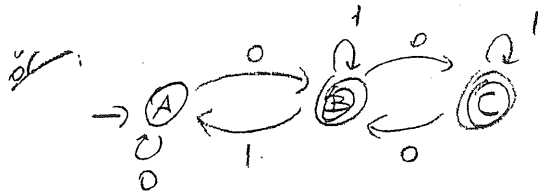
$$B = A0 \quad (2)$$

$$A = \lambda + A0 + A01 = \lambda + A(0+01) \quad (3)$$

$$R = \frac{\lambda}{0} \frac{0}{1} \frac{0}{P}$$

$$A = \lambda(0+01)^* = (0+01)^*$$

$$B = A0 = (0+01)^* 0. \text{ ugr durumu B old. denli } T(M) = (0+01)^* 0$$



$$A = \lambda + A0 + B1$$

$$B = A0 + B1 + C0$$

$$C = B0 + C1$$

$$R = \frac{\lambda}{0} \frac{0}{1} \frac{0}{P}$$

$$C = B01^*$$

$$B = A0 + B1 + B01^*0 = A0 + B(1+01^*0) = \boxed{A0(1+01^*0)^*}$$

$$Q + R P$$

$$A = \lambda + A0 + A0(1+01^*0)^*1 = \lambda + A(0+0(1+01^*0)^*1)$$

$$R = \frac{\lambda}{0} \frac{0}{1} \frac{0}{P}$$

$$A = (0+0(1+01^*0)^*1)^* \quad B = (0+0(1+01^*0)^*1)^* 0(1+01^*0)^*$$

$$C = (0+0(1+01^*0)^*1)^* 0(1+01^*0)^* 01^*$$

$$T(M) = B+C = B+B01^* = B(\lambda+01^*) = \boxed{(\lambda+01^*)}$$

Bölüm 3 DİL BİLGİSİ VE DİLLER

Hafta 5-6

$A, B, C \dots$	sözdizim değişkenleri
$a, b, c, \dots, 0, 1, 2$	u _g simgeleri
U, V, W, X, Y, Z	sözdizim ya da u _g simge
u, v, w, x, y, z	u _g simge dizileri
$\alpha, \beta, \delta \dots$	tümcesel yapılar

(10)

3.1. Dilbilgisi ve Dilin B.imsel Tanımı

$G = \langle V_N, V_T, P, S \rangle$ şeklinde b.imsel olarak bir dil tanımlanabilir.

V_N : Sözdizim değişkenleri kümesi (sonlu küme)

V_T : u_g simgeleri kümesi (sonlu küme)

S : Başlangıç değişkeni $S \in V_N$

P : Yeniden türetim kuralları

$\alpha \Rightarrow \beta$: " α yerine β yazılabilir" $\alpha \in V^+$ $V = V_N \cup V_T$
 $\beta \in V^+$ $V^+ = V^+ - \{\alpha \mid \alpha \in V\}$

$$L(G) = \{ w \mid w \in V_T^+, S \Rightarrow w \}$$

$S \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \alpha_3 \dots \Rightarrow \alpha_n \Rightarrow w$ $\alpha_1, \alpha_2, \alpha_3 \dots \alpha_n$: Tümcesel yapı
 w : Tümce

Örnek

$$G_{3.1} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{ S \}$$

$$V_T = \{ 0, 1 \}$$

$$P : S \Rightarrow 0S1$$

$$S \Rightarrow 01$$

dilbilgisi veriliyor. Bu dilbilgisi tarafından türetilen tümce-lerden bazılarını bulalım.

$$S \Rightarrow 01$$

$$S \Rightarrow 0S1 \Rightarrow 0011$$

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$$

$$L(G) = \{ 0^n 1^n \mid n \geq 1 \}$$

3.2. Dillerin Sıfılandırılması

Dilbilgisi ve türettikleri diller, yeniden yazma kurallarına göre 4 sınıfa ayrılır.

Tür-0 Bağımsız Dilbilgisi ve diller

Tür-1 Bağımlı-bağımlı " " "

Tür-2 Bağımlı-bağımsız " " " $\rightarrow$ Context Free

Tür-3 Düzenli Dilbilgisi ve diller $\rightarrow$ Regular

3.2.1. Tür-0 Dilbilgisi ve Dil

Kısıtlanmış $\Rightarrow$ Recursively Enumerable DL.
Dilbilgisi $\Rightarrow$ Sayılabilir

Yeniden yazma kuralları $\alpha \Rightarrow \beta : \alpha \in V^+, \beta \in V^*$ şeklinde olan dillerden

ÖRNEK $G_{3,2} = \langle V_N, V_T, P, S \rangle$

Bu dil tarafından türetilen cümlelerden bazıları:

$$V_N = \{S, L, R, A, B, C\}$$

$$V_T = \{a\}$$

$$S \Rightarrow LAaR \Rightarrow LaaAR \Rightarrow LaaC \Rightarrow LaCa \Rightarrow LCaa \Rightarrow \underline{aa}$$

$$P: S \Rightarrow LAaR$$

$$Aa \Rightarrow aaA$$

$$aB \Rightarrow Ba$$

$$LB \Rightarrow LA$$

$$aC \Rightarrow Ca$$

$$LC \Rightarrow \lambda$$

$$AR \Rightarrow BR|C$$

$$S \Rightarrow LAaR \Rightarrow LaaAR \Rightarrow LaaBR \Rightarrow LaBaR \Rightarrow LBaR$$

$$\Rightarrow LAaR \Rightarrow LaaAR \Rightarrow LaaAR \Rightarrow LaaAR$$

$$\Rightarrow LaaBR \Rightarrow LaaCaa \Rightarrow LaCaar \Rightarrow LCaaar \Rightarrow \underline{aaar}$$

$$L(G_{3,2}) = \{a^i \mid i=2^k, k \geq 1\} = (G_{3,2} \text{ dilinin tanımı})$$

3.2.2. Tür-1 Dilbilgisi ve Dil

Bağımlı Bağımsız : Özgün : Recursive

Tür-1 dilbilgisinin yeniden yazma kuralları

$$\alpha \Rightarrow \beta : \alpha \in V^+, \beta \in V^*, |\alpha| \leq |\beta| \text{ olarak şartlarla tanımlanabilir}$$

Bu dilbilgisinin yeniden yazma kurallarının tümü

$\alpha_1 A \alpha_2 \Rightarrow \alpha_1 \beta \alpha_2$ $A \in V_N, \alpha_1, \alpha_2, \beta \in V^*$ biçiminde olan bir normal biçime dönüştürülebilir. Bu normal biçimde $\alpha_1 \dots \alpha_2$ bağlamında A 'nın yerini β konulabilir. Bu dilbilgisi bağlama bağımlı olarak adlandırılır.

ÖRNEK $G_{3,3} = \langle V_N, V_T, P, S \rangle$

Bu dilden türetilen bazı cümleler bulalım:

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b, c\}$$

$$S \Rightarrow aAB \Rightarrow abB \Rightarrow \underline{abc}$$

$$S \Rightarrow aSAB \Rightarrow aaABAB \Rightarrow aabBAB \Rightarrow aabbBB$$

$$\Rightarrow aabbcB \Rightarrow \underline{aabbcc}$$

$$P: S \Rightarrow aSAB$$

$$S \Rightarrow aAB$$

$$BA \Rightarrow AB$$

$$aA \Rightarrow ab$$

$$bA \Rightarrow bb$$

$$bB \Rightarrow bc$$

$$cB \Rightarrow cc$$

$$G_{3,3} \text{ dilbilgisinin tanımı } L(G_{3,3}) = \{a^n b^n c^n \mid n \geq 1\}$$

3.2.3 Tür-2 Dilbilgisi ve DL - Context Free - Bağlılıktan Bağlıdır

Tür-2 dilbilgisinin yeniden yazma kuralları

$A \Rightarrow \beta$: $A \in V_N$ $\beta \in V^*$ şeklinde sol tarafta tek bir

A değişkeni yer alır. Hangi bağlamda olursa olsun A yerine β konulabilir. Bu nedenle Tür-2 dilbilgisi bağlamdan bağımsız dilbilgisi ile denir. $G_{3.1}$ di ki Tür-2 dilbilgisi ne girer.

ÖRNEK: $G_{3.4} = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B\}$

$V_T = \{a, b\}$

P: $S \Rightarrow aB | bA$

$A \Rightarrow a | aS | bAA$

$B \Rightarrow b | bS | aBB$

Bu dil tarafından türetilen bazı kelimeler

$S \Rightarrow bA \Rightarrow baS \Rightarrow babaA \Rightarrow \underline{babab}$

$S \Rightarrow aB \Rightarrow abS \Rightarrow abbaA \Rightarrow abbaaS \Rightarrow abbaaB$
 $\Rightarrow \underline{abbaaab}$

$S \Rightarrow bA \Rightarrow bbaA \Rightarrow bbaSA \Rightarrow bbaaBA \Rightarrow bbaaBSA$
 $\Rightarrow \underline{bbaaBaBA} \Rightarrow bbaaBabaA \Rightarrow bbaaBabab$

Erit sayıda a ve b geçen diğer kelimeler.

$L(G_{3.4}) = \{a^n b^n | n \geq 1\}$

3.2.4. Tür-3 Dilbilgisi ve DL - Düzenli Dil - Regüler

Bu Tür-3 dilbilgisinin yeniden yazma kuralları:

$A \Rightarrow aB$

$A \Rightarrow a$

$A \Rightarrow \lambda$: $A, B \in V_N$, $a \in V_T$ şeklinde sol tarafta tek bir

değişken, sağ tarafta ise ya tek bir a simge ile bir değişken, ya bir a simge ya da λ geçeri yer alır.

ÖRNEK: $G_{3.5} = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B\}$

$V_T = \{0, 1\}$

P: $S \Rightarrow 0S | 0A | 0\lambda$

$A \Rightarrow 0B$

$B \Rightarrow 1S$

Bu dilden türetilen bazı kelimeler.

$S \Rightarrow 0S \Rightarrow 00$

$S \Rightarrow 0A \Rightarrow 00B \Rightarrow 001S \Rightarrow 001$

$S \Rightarrow 0S \Rightarrow 00A \Rightarrow 000B \Rightarrow 0001S \Rightarrow 00010A$

$\Rightarrow 000100B \Rightarrow 0001001S \Rightarrow 00010010$

$\{0, 1\}$ alfabesinde n tane her 1'den önce en az 2 tane (s.f.) 0 bulunması gerekir.

3.2.5. Sağ Doğrusal ve Sol Doğrusal Dilbilgisi

Sağ Doğrusal Dilbilgisi: Yeniden yazma kuralları

$$A \Rightarrow wB$$

$A \Rightarrow w$: $A, B \in V_N$, $w \in V_T^*$ biçiminde olan dilbilgisine

sağ doğrusal dilbilgisi denir. Sol tarafında bir değişken, sağ tarafında ise bir w simgeler dizisi ile bir değişken ya da bir w simgeler dizisi yer alır.

Sol Doğrusal Dilbilgisi:

$$A \Rightarrow BW$$

$A \Rightarrow w$: $A, B \in V_N$, $w \in V_T^*$ biçiminde olan dilbilgisine

sol doğrusal dilbilgisi denir. Sol tarafında bir değişken, sağ tarafında ise bir w simgeler dizisi veya bir değişken ~~ya da~~ ile bir w simgeler dizisi yer alır.

NOT: Her Tür-3 dilbilgisi aynı zamanda sağ doğrusaldır.

$$A \Rightarrow aabB \quad \text{veya} \quad A \Rightarrow aab \quad \text{olabilir.}$$

Sağ doğrusal dilbilgisi ek değişkenlerle Tür-3 dilbilgisine dönüştürülebilir.

$$A \Rightarrow a_1 a_2 \dots a_n B \quad \text{ise} \quad \begin{cases} A \Rightarrow a_1 A_1 \\ A_1 \Rightarrow a_2 A_2 \\ \vdots \\ A_{n-1} \Rightarrow a_n B \end{cases}$$

$$A \Rightarrow a_1 a_2 \dots a_n \quad \text{ise} \quad \begin{cases} A \Rightarrow a_1 A_1 \\ A_1 \Rightarrow a_2 A_2 \\ \vdots \\ A_{n-1} \Rightarrow a_n \end{cases} \quad \text{olur.}$$

$$\text{ör.} / A \Rightarrow aabB \quad \text{ise} \quad \begin{cases} A \Rightarrow a_1 A_1 \\ A_1 \Rightarrow a_2 A_2 \\ A_2 \Rightarrow bB \end{cases}$$

sağ doğr.

$\equiv$ Tür-3. kuralları

ÖRNEK: $G_{3,6} = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B\}$

$V_T = \{0, 1\}$

$P: S \Rightarrow 0A$

$A \Rightarrow 10A/1$

$G'_{3,6}$ S-1 dşrusal

$G'_{3,6} = \langle V_N, V_T, P, S \rangle$

$V_N = \{S\}$

$V_T = \{0, 1\}$

$P: S \Rightarrow S1010$

Bu dilbilgi sağ doğrusaldır.

$S \Rightarrow 0A \Rightarrow 0$

$S \Rightarrow 0A \Rightarrow 010A \Rightarrow 010$

$S \Rightarrow 0A \Rightarrow 010A \Rightarrow 01010A \Rightarrow 01010$

$L(G_{3,6}) = 0(10)^* = (01)^*0$

$G''_{3,6}$ tur-3 denlişir.

$G''_{3,6} = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B\}$

$V_T = \{0, 1\}$

$P: S \Rightarrow 0A$

$A \Rightarrow 1B/1$

$B \Rightarrow 0A$

(12)

3.3. Düzgün DL- Sonlu Özdevimliklik.

3.3.1. Düzgün Dilin Türettirici DL Tanıyan Sonlu Özdevimin Bulunması=

* Bir düzgün dilbilgi verildiğinde (tur-3), bu dilbilginin türettirici dil tanıyan bir sonlu özdevim bulunabilir.

Tur-3 Dilbilginin Türettirici DL Tanıyan Sonlu Özdevimin Bulunması=

$T(M) = L(G)$ eşitliğini sağlayan M özdevimi bulunurken

$G = \langle V_N, V_T, P, S \rangle$

$M = \langle Q, \Sigma, q_0, \delta, F \rangle$

$Q = V_N \cup \{C\}$

$\Sigma = V_T$

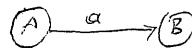
$q_0 = S$

$F = \{C\}$

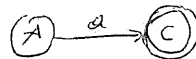
$= \{C, S\}$

: eğer $L(G)$ λ içeriyorsa
: eğer $L(G)$ λ içeriyorsa

$\delta: A \Rightarrow aB$ için $\delta(A, a) = B$



$A \Rightarrow \lambda$ için $\delta(A, \lambda) = C$



2.200/1

$$G_{3.7} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 1A$$

$$A \Rightarrow 0B$$

$$B \Rightarrow 0B \mid 1S \mid 1$$

$$M = \langle Q, \Sigma, q_0, F, \delta \rangle$$

$$Q = V_N \cup \{C\} = \{S, A, B, C\}$$

$$q_0 = \{S\}$$

$$\Sigma = V_T = \{0, 1\}$$

$$F = \{C\}$$

$$\delta: \delta(S, 0) = S$$

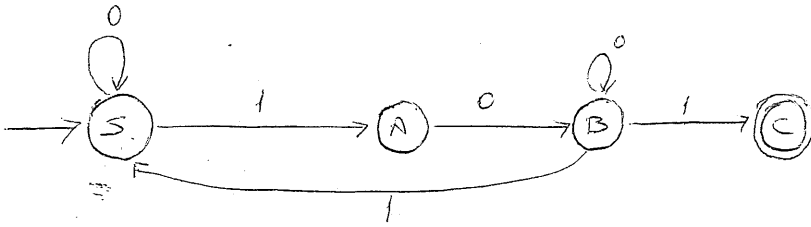
$$\delta(A, 0) = B$$

$$\delta(B, 1) = S$$

$$\delta(S, 1) = A$$

$$\delta(B, 0) = B$$

$$\delta(B, 1) = C$$



● Sağ Döğrusal Dilbilgisinin Türettiği Dilin Tanıyan Sonlu Özdevimin Bulunması

Bir $L(G)$ sağ döğrusal dilin tanıyan $T(M)$ bulunurken;

$$G = \langle V_N, V_T, P, S \rangle$$

$$M = \langle Q, \Sigma, q_0, \delta, F \rangle$$

$$Q = \{[S], [\lambda], [\alpha_i], [\alpha_i], [\alpha_i], \dots\} \quad \alpha_i: \text{sonak, suffix}$$

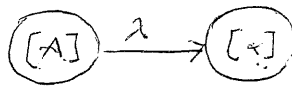
$$F = [\lambda]$$

$$\Sigma = V_T$$

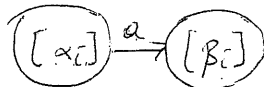
$$q_0 = [S]$$

δ : Durum geçişleri yeniden yazma kurallarından ve bu kuralların sağ taraflarının son eklerinden:

$$\bullet A \Rightarrow \alpha$$



• İlk simgesi bir us simge olan her $\alpha_i (\alpha_i = a\beta_i)$ için



şeklinde türetilir.

ÖRNEK

$$G_{3,3} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{0, 1\}$$

$$P: S \Rightarrow 0S \mid 1S \mid 111A$$

$$A \Rightarrow 0A \mid 1A \mid \lambda$$

$$M = \langle Q, \Sigma, q_0, \delta, F \rangle$$

$$Q = \{[S], [\lambda], [0S], [1S], [111A], [11A], [1A], [A], [0A]\}$$

$$F = \{\lambda\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = \{S\}$$

13

δ : Önce sol taraftaki sözün değişkenlerinin λ geçişleri yapılır.

$$\delta([S], \lambda) = [0S]$$

$$\delta([A], \lambda) = [0A]$$

$$\delta([S], 1) = [1S]$$

$$\delta([A], 1) = [1A]$$

$$\delta([S], \lambda) = [111A]$$

$$\delta([A], \lambda) = [\lambda]$$

İkinci olarak sağ tarafın $\Sigma = \{0, 1\}$ geçişleri yapılır.

$$\delta([0S], 0) = [S]$$

$$\delta([111A], 1) = [11A]$$

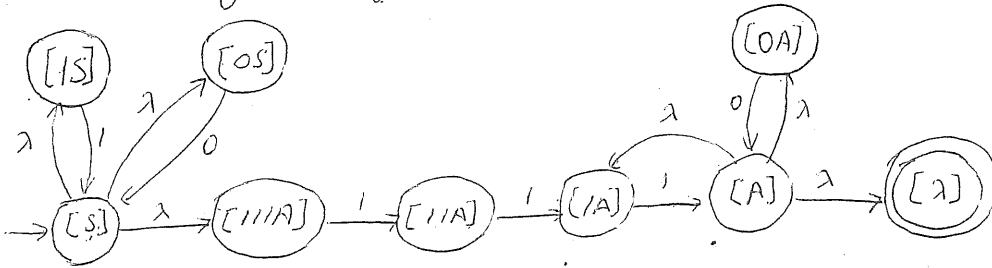
$$\delta([1A], 1) = [A]$$

$$\delta([1S], 1) = [S]$$

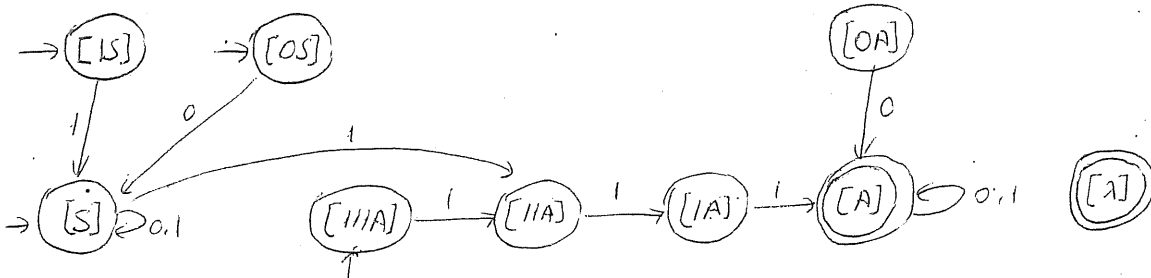
$$\delta([11A], 1) = [1A]$$

$$\delta([0A], 0) = [A]$$

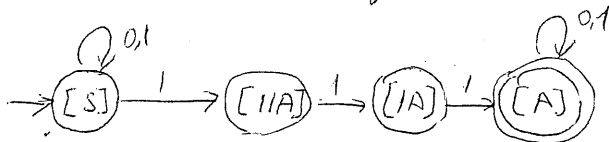
Örneklerin geçiş diyagramı:



λ 'lar yok edildiğinde.



gereksiz durumlar çıkarıldığında.



4. G^2 deki tüm genlerden gazma kurallarının sağ tarafları tırs çevrilerek elde edilir.

3. M^2 Sch-

- 11 ediz. edizioni

Bu dil sor doğrudur. Bu dile tınan M

Makinesiir adan adan bulakim.

P: $S \Rightarrow 0/5/0$

$P^R: S \Rightarrow 0/015$

$$G^2 = \{[s], [\lambda], [0], [01s], [1s]\}$$

$$\begin{aligned} \delta^R: \delta^R([S], \lambda) &= [0] \\ \delta^R([S], \lambda) &= [015] \end{aligned}$$

$$\begin{aligned} s^2([01], 0) &= [15] & s^2([0], 0) &= [2] \\ s^2([15], 1) &= [5] \end{aligned}$$

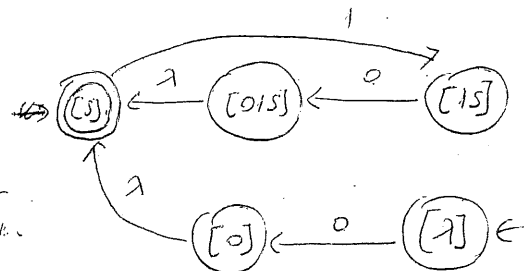
3.

```
graph LR; start(( )) --> s((s)); s -- 1 --> os((o/s)); os -- 0 --> is((is)); is -- 1 --> s; s -- 1 --> o((o)); o -- 0 --> lambda(((λ))); style start fill:none,stroke:none; style lambda fill:none,stroke:none;
```

→

$F.S \rightarrow S.S$

$S.S \rightarrow F.S$ 4 types
other Test scripts



```

graph LR
    start(( )) --> C((C))
    C -- 1 --> B((B))
    B -- 0 --> C
    A((A)) -- 1 --> B
    S(((S))) -- 0 --> A
    S -- 0 --> S
    style start fill:none,stroke:none
    style C stroke-width:4px
    style S stroke-width:4px
  
```

3.32. Sonlu Ödevinin Tanımı. D6 Türetken Düzgün Döndürülen Bulunması.

Bir sonlu ödevinin verildiğinde bu $L(G) = T(M)$ eşitliğini sağ-
layan tür-3 dilbilgisi aşağıdaki gibi bulunur.

$$M = \langle Q, \Sigma, q_0, \delta, F \rangle$$

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = Q$$

$$V_T = \Sigma$$

$$S = q_0$$

S : Her $(A) \xrightarrow{a} (B)$ için $A \Rightarrow aB$ (14)

: Her $(A) \xrightarrow{a} (C)$ için $A \Rightarrow aC$ ve $A \Rightarrow a$

: Eğer başlangıç durumu bir us durursa: $S \Rightarrow \lambda$

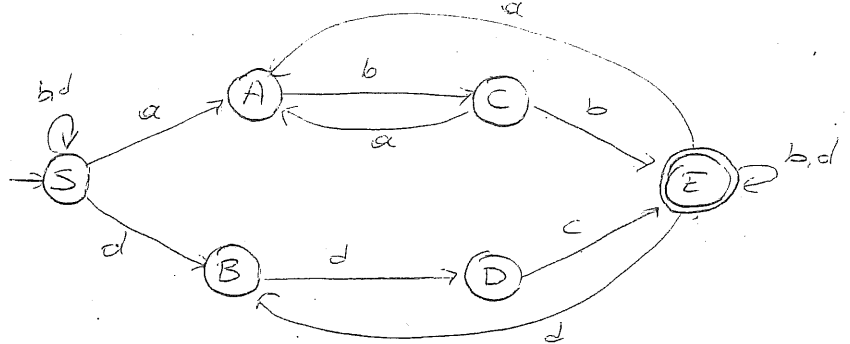
$\Rightarrow (S)$

ÖRNEK $M = \langle Q, \Sigma, \delta, q_0, F \rangle$

$$Q = \{S, A, B, C, D, E\}$$

$$V_T = \{a, b, c, d\}$$

$$F = \{E\}$$



$L(G)$:

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = Q = \{S, A, B, C, D, E\}$$

$$V_T = \Sigma = \{a, b, c, d\}$$

$$P: S \Rightarrow aA | bS | dS | dB$$

$$A \Rightarrow bC$$

$$B \Rightarrow dD$$

$$C \Rightarrow aA | bE | b$$

$$D \Rightarrow cE | c$$

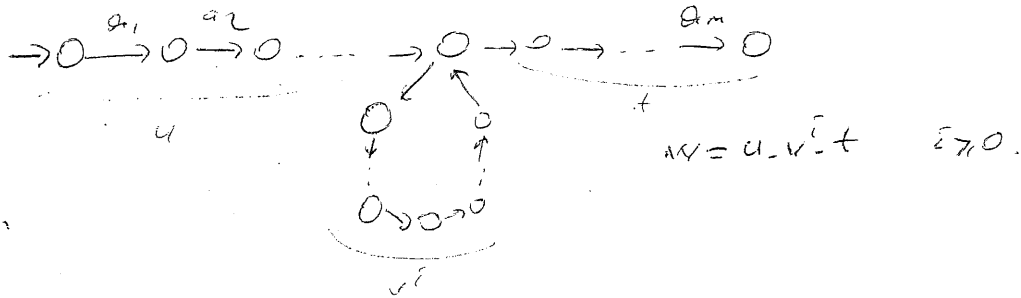
$$E \Rightarrow aA | bE | b | dE | d | dB$$

şeklinde bulunur.

3.4. Pumping Lemma

L dögün bir dil olsun (tür-3). L 'yi tanıyan M sonlu otomatik bir durum sayısı n olsun. Eğer L 'nin özelliklerinden birisi n den büyükse u, w, v yi tanımlayarak bir durumdan diğer geçebilir.

$w = a_1 a_2 a_3 \dots a_m$ $|w| = m > n$ w 'ye karşılık gelen yol mutlaka döngülerden



Her dögün dil için şöyle bir tanımlama (n) vardır ki, eğer bu dilin uzunluğu bu değerden büyük bir tamsayı (w) varsa bu tamsayı

$w = uvt$ biçiminde yazılabilir ve dil

$w = uv^i t$ $i \geq 0$ biçimindeki tüm tamsayılar geçer.

L_1 ve L_2 dögün diller ise $L_1 \cup L_2$
 $L_1 \cap L_2$ de dögün dillerdir.
 L_1'

$L_1' = \Sigma^* - L_1$ olarak tanımlanır.

BÖLÜM 4- BAĞLAMDAKİ - BAĞIMSIZ DİL BİLGİSİ VE DİLİLER

4.1. Bağlamdan Bağımsız Dilbilgisi (Context-Free) Tür-2:

Bağlamdan bağımsız dilbilgisi bir dökü olarak tanımlanabilir

$$G = \langle V_N, V_T, P, S \rangle$$

- V_N : Sözcük Değişkenleri
- V_T : Uç Sözcükler Kümesi
- S : Başlangıç Kümesi
- P : $A \Rightarrow B$ $A \in V_N$ $B \in V^*$

(15)

ÖRNEK

$$G_{4.1} = \langle V_N, V_T, P, S \rangle$$

- $V_N = \{S\}$
- $V_T = \{+, -, *, /, (,), v, c\}$
- $P: S \Rightarrow S+S \mid S-S \mid S*S \mid S/S \mid (S) \mid v \mid c$

$G_{4.1}$ dilbilgisinden türetilen bazı türetilenler:

- $S \Rightarrow S*S \Rightarrow S^*(S) \Rightarrow S^*(S-S) \Rightarrow v^*(S-S) \Rightarrow v^*(v-S) \Rightarrow \underline{v^*(v-c)}$
- $S \Rightarrow S*S \Rightarrow (S)^*S \Rightarrow (S)^*(S) \Rightarrow (S+S)^*(S) \Rightarrow (S+S)^*(S) \Rightarrow (S+S)^*(S-S) \Rightarrow (S+S)^*$
 $(S-S/S) \Rightarrow (v+S)^*(S-S/S) \Rightarrow (v+c)^*(S-S/S) \Rightarrow (v+c)^*(v-S/S) \Rightarrow (v+c)^*(v-v/S)$
 $\Rightarrow \underline{(v+c)^*(v-v/c)}$

Bu dil, $L(G)$, temel aritmetik işlemler, değişkenler, değişkenler ve parantezlerden oluşan bir dildir.

- $S \Rightarrow S+S \Rightarrow S*S+S \Rightarrow (S)^*S+S \Rightarrow (S/S)^*S+S \Rightarrow (S/(S))^*S+S \Rightarrow (S/(S-S))^*S+S$
 $\Rightarrow (v/(S-S))^*S+S \Rightarrow (v/(v-S))^*S+S \Rightarrow (v/(v-c))^*S+S \Rightarrow (v/(v-c))^*v+S$
 $\Rightarrow \underline{(v/(v-c))^*v+c}$

42. Türetme Ağacı

Bir dilbilgisi tarafından türetilen tümcesel yapı ve tümceler aşağıdaki gibidir.

$$S \Rightarrow \underbrace{\beta_1 \Rightarrow \beta_2 \Rightarrow \beta_3 \dots \Rightarrow \beta_{n-1} \Rightarrow \beta_n}_{\text{tümcesel yapı}} \Rightarrow \underbrace{w}_{\text{tümce}} \quad \beta_i \in V^* \quad w \in V_T$$

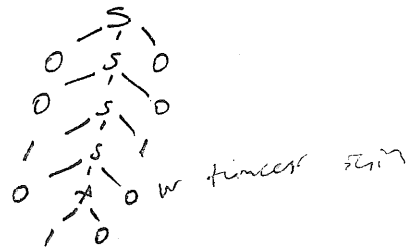
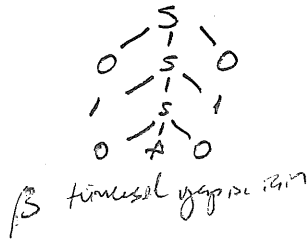
(16)

Context free dillerin her tümcesel yapısına ve tümcesine bir türetme ya da ayrıştırma ağacı denk gelir.

$$\beta = 010A010 \\ w = 0010100100$$

$$\beta = 010A010$$

$$w = 0010100100$$



Türetme (Ayrıştırma) Ağacının Tanımı =

- Ağacın kök etiketi S' tir.
- Kök dışındaki düğüm etiketleri sözdizim değişkenleridir. ($A \in V_N$)
- Eğer ağacın bir tümcesel yapıya karşılık geliyorsa yaprakların etiketleri ~~ya da~~ ^{söz dizim veya} uz string olabilir. Tümceye karşılık gelen etiketler u string $A \in V_T$
- Eğer bir düğümün etiketi A ise bu düğüm bir u düğüm olmalı ve kardeşi bulunmamalıdır.
- Eğer A aradığımız altında sağda $x_1, x_2, \dots, x_k$ ifade varsa dilbilgimizde $A \Rightarrow x_1 x_2 \dots x_k$ kuralı olmalı.

Soldan ve Sağdan Türetme

- Eğer bir dilin bir tümcesine birden çok türetme ağacı karşılık geliyorsa dil, belirsiz olmaya sahiptir. Belirsizlik, dilbilgisi ve dilin temel özellikleridir.
- Eğer bir tümce türetilirken, her adımda en soldaki değişkene bir türetme uygulanıyorsa, yapılan türetmeye soldan türetme denir.
 - Eğer bir tümce türetilirken, her adımda en sağdaki değişkene bir türetme uygulanıyorsa, yapılan türetmeye sağdan türetme denir.

$L(G_{4.1})$ dilinin iki tümcesine karşılık gelen türetme ağacı aşağıdadır.

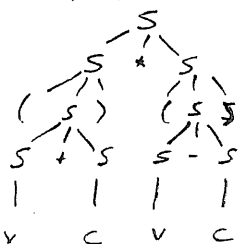
$$w_1 = (v+c)^*(v-c)$$

w_1 i soldan türetilim:

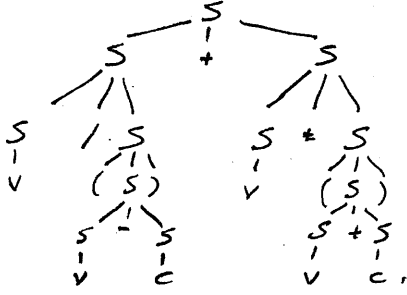
$$\begin{aligned} S &\Rightarrow S^* S \Rightarrow (S)^* S \Rightarrow (S+S)^* S \Rightarrow (v+c)^* S \Rightarrow (v+c)^* (S-S) \\ &\Rightarrow (v+c)^* (S-S) \Rightarrow (v+c)^* (v-S) \Rightarrow (v+c)^* (v-c) \end{aligned}$$

sağdan türetilim:

$$\begin{aligned} S &\Rightarrow S^* S \Rightarrow S^* (S) \Rightarrow S^* (S-S) \Rightarrow S^* (S-c) \Rightarrow S^* (v-c) \\ &\Rightarrow (S)^* (v-c) \Rightarrow (S+c)^* (v-c) \Rightarrow (v+c)^* (v-c) \end{aligned}$$



$$W_2 = v/(v+c) + v^*(v+c)$$



W_2 soldan türetilirse

$$\begin{aligned} S &\Rightarrow S+S \Rightarrow S/S+S \Rightarrow v/S+S \Rightarrow v/(S)+S \\ &\Rightarrow v/(S-S)+S \Rightarrow v/(v-S)+S \Rightarrow v/(v-c)+S \\ &\Rightarrow v/(v-c)+S^*S \Rightarrow v/(v-c)+v^*S \\ &\Rightarrow v/(v-c)+v^*(S) \Rightarrow v/(v-c)+v^*(S+S) \\ &\Rightarrow v/(v-c)+v^*(v+c) \Rightarrow \underline{v/(v-c)+v^*(v+c)} \end{aligned}$$

$$\begin{aligned} W_2 \text{ sağdan türetilirse: } S &\Rightarrow S+S \Rightarrow S+S^*S \Rightarrow S+S^*(S) \Rightarrow S+S^*(S+S) \\ &\Rightarrow S+S^*(S+c) \Rightarrow S+S^*(v+c) \Rightarrow S+v^*(v+c) \Rightarrow S/S+v^*(v+c) \\ &\Rightarrow S/(S)+v^*(v+c) \Rightarrow S/(S-S)+v^*(v+c) \Rightarrow S/(S-c)+v^*(v+c) \Rightarrow S/(v-c) \\ &+v^*(v+c) \Rightarrow \underline{v/(v-c)+v^*(v+c)} \end{aligned}$$

4.3. Dilbilgisinin Yalınlaştırılması

Verilen bir CFL' in daha az sayıda değişken ve kural içeren ve belirli biçimdeki kuralları içermeyen bir dilbilgisine dönüştürülmesine dilbilgisinin yalınlaştırılması denir.

Bağımlardan bağımsız bir dilbilgisi yalın olmalıdır.

4.3.1. Özüneltir Kuralı Özüneltir Değişken

Bir yeniden yazma kuralının solundaki değişken, kuralın sağında da yer alıyorsa, $A \Rightarrow \alpha_1 A \alpha_2$ biçiminde ise bu kurala özüneltir kural, A değişkenine de özüneltir değişken denir. Eğer ilk singe ise $A \Rightarrow A \alpha_2$ şeklinde bu kurala doğrudan özüneltir kural denir.

Bu durumlarda başlangıç değişkeni S ' in özüneltir olması istenmez.

$$S \Rightarrow S \text{ ise } \begin{aligned} S &\Rightarrow S' \\ S' &\Rightarrow S \end{aligned} \text{ şeklinde dönüşüm yapılır.}$$

4.3.2. Yok Edilebilir Değişken : Eğer bir değişken yerine, bir ya da birkaç türetme sonunda λ konulabiliyorsa

$A \Rightarrow \lambda$ ya da $A^* \Rightarrow \lambda$ değişkenine yok edilebilir değişken denir.

Bazı işlemlerde sadece $S \Rightarrow \lambda$ dışında λ kuralının bulunmaması istenir. Yani dil λ ' ı içermiyorsa ($\lambda \notin L$) dilbilgisinde λ bulunmayacaktır. Eğer dilde λ varsa tek λ kuralı $S \Rightarrow \lambda$ olacaktır.

Algoritma 4.1. Yok Edilebilir Değişkenlerin Bulunması.

1. $T_L = \{A \mid (A \Rightarrow A) \in P\};$
2. $Test_i = \emptyset;$
3. $(T_L = Test_i)$ oluncaya kadar aşağıdaki işlemler tekrarlar:
 - 3.1. $Test_i = T_L;$
 - 3.2. $(\forall \alpha)$ için her değişken (A) için aşağıdaki işlemler:

Eğer α bir $(A \Rightarrow \alpha)$ kuralı için $\alpha \in (T_L)^*$ ise A 'yı (T_L) 'ye ekle;

(17)

son 3.1.

son 3

Algoritma 4.2. Başlangıç Değişkeni dışında yok edilebilir değişken bulunma ve değer dilbilgisinin bulunması

1. P' deki tüm $(A \Rightarrow \alpha)$ kurallarını çıkar.
2. Eğer $\alpha \in L(G)$ ise P' ye $(S \Rightarrow \alpha)$ kuralını ekle;
3. P' deki her $(A \Rightarrow \alpha)$ kuralı için aşağıdaki işlemler:
 - 3.1. Eğer α yok edilebilir değişken içeriyorsa, bu değişkenlerin bir ya da birkaçını α 'dan çıkararak oluşturulabilen her α_i için aşağıdaki işlemler:

P' ye $(A \Rightarrow \alpha_i)$ kuralını ekle

son 3.1.

son 3

örnek

$$G_{4.5} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b\}$$

$$P : S \Rightarrow ACA$$

$$A \Rightarrow B \mid C \mid \alpha A \alpha$$

$$B \Rightarrow bB \mid b$$

$$C \Rightarrow cC \mid \lambda$$

$G_{4.5}$ e eşdeğer başlangıç değişkeni dışında yok edilebilir değişken içermeyen bir dilbilgisi bulmak için önce dilbilgisinin 4.1'e göre yok edilebilir def. bulunması.

1. $T_L = \{C\}$; Çünkü $C \Rightarrow \lambda$ dir.
2. $Test_i = \emptyset$
3. $(Test_i = T_L)$ olmadığı sürece.
 - 3.1. $Test_i = T_L = \{C\}$

C 'ye giden değişkenler : A A T_L 'ye eklenir.

$A \Rightarrow C$ $T_L = \{A, C\}$
 - 3.2. $Test_i \neq T_L$

3.1. $Test_i = T_L = \{A, C\}$ A ya giden değişkenler T_L^* a bakılır.

ACA ya giden $S \Rightarrow ACA$ sağlanır. S T_L 'ye eklenir.

$T_L = \{S, A, C\}$
 - 3.3. $Test_i \neq T_L$

3.1. $Test_i = T_L = \{S, A, C\}$ daha yok $T_L = \{S, A, C\}$
3. $Test_i = T_L$ olur değişken çık $T_L = \{S, A, C\}$

$T_L = \{S, A, C\}$ olarak bulundu.

Dilbilgisinin başlangıç değişkeni yok edilebilir bir değişken olduğuna göre dilbilgisinin türettiği dil λ 'yı içermektedir. Bu nedenle eşdeğer dilde $S \Rightarrow \lambda$ kuralı yer alacaktır.

Alg. 4.2. kullanılarak S dışında yok edilebilir değişken içermeyen dilini bulalım.

1. $C \Rightarrow \lambda$ çıkarıldı.

2. P 'ye $S \Rightarrow \lambda$ eklendi.

3. Her $A \Rightarrow \alpha$ için

3.1. (eğer T_L içermeyen bir α da bir T_L içermeyen alt kime alınırsa)
Her α_i için $A \Rightarrow \alpha_i$ şeklinde ekle)

$S \Rightarrow ACA$

$S \Rightarrow \lambda$: 2'de eklendi ve $C \Rightarrow \lambda$ çıkarıldı.

$A \Rightarrow B$

$A \Rightarrow C$

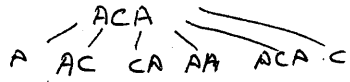
$A \Rightarrow aAa$

$B \Rightarrow bB$

$B \Rightarrow b$

$C \Rightarrow cC$

$C \Rightarrow \lambda$



$S \Rightarrow ACA|A|C|AC|CA|AA|\lambda$

$A \Rightarrow B|C|aAa|aa$

$B \Rightarrow bB|b$

$C \Rightarrow cC|c$

*Sadece T_L çıkarılır.

Bu durumda türetilen dil:

$G_{4.5} = \langle V_N, V_T, P, S \rangle$ $V_N = \{S, A, B, C\}$ $V_T = \{a, b\}$

P : $S \Rightarrow ACA|AC|CA|AA|A|C|\lambda$

$A \Rightarrow B|C|aAa|aa$

$B \Rightarrow bB|b$

$C \Rightarrow cC|c$

4.3.3. Birim Türetme Kuralları =

$A \Rightarrow B$ biriminde olan yeniden yazma kuralına birim türetme ya da zincir kuralı denir.

Bağılandıran bağımsız bir dilbilgisi verildiğinde, bu dilbilgisine eşdeğer birim türetme kuralları içermeyen bir dilbilgisi bulunabilir. Öncelikle T_A (her A) değişkeni için, bu değişkenden türetilen diğer değişkenler kümesi) bulunmalıdır.

$\forall A \in V_N \Rightarrow T_A = \{B \mid B \in V_N, A \Rightarrow B\}$ Her değişkenden türetilen diğer değişkenler kümesi alg. 4.3. ile bulunabilir.

Algoritma 4.3.

(18)

1. $T_A = \{A\}$

2. $T_{\text{Eski}} = \emptyset$

3. ($T_A = T_{\text{Eski}}$) oluncaya kadar.

3.1. $T_{\text{Yeni}} = T_A - T_{\text{Eski}};$

3.2. $T_{\text{Eski}} = T_A$

3.3. (T_{Yeni}) deki her B değişkeni için

3.3.1. Her $(B \Rightarrow C)$ kuralı için

$$T_A = T_A \cup \{C\};$$

son.3.3.1.

son.3.3.

son.3.

Algoritma 4.4. Birim Türetme Kuralı İçermeyen Dilbilgisinin Bulunması.

01. $P = P - \{A \Rightarrow B \mid A, B \in V_N\}$

2. V_N deki her A değişkeni için Eğer T_A da A dan başka en az bir değişken varsa

2.1. $T_A = T_A - \{A\}$

2.2. T_A 'da her B değişkeni için

B . kurallarının tümü $B \Rightarrow \beta_1/\beta_2/\beta_3/\dots/\beta_k$ olmak üzere

$$P = P \cup \{A \Rightarrow \beta_1/\beta_2/\beta_3/\dots/\beta_k\};$$

son.2.2.

son.2.

ÖRNEK $G_{4.6} = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow A/AA/AC/CA/ACA/\lambda$$

$$A \Rightarrow B/aAa/aa$$

$$B \Rightarrow C/bB/b$$

$$C \Rightarrow cC/c$$

Bu dile eşdeğer birim türetme kuralları içermeyen bir dilbilgisi bulmak için önce A, B, C, S değişkenlerinden türetilen değ. kümelerinin bulunması gerekir.

$$\text{Alg. 4.3 ile } T_S = \{S, A, B, C\} \quad T_B = \{B, C\}$$

$$T_A = \{A, B, C\} \quad T_C = \{C\}$$

Algoritma 4.4 Kullanılarak

$$G'_{4.6} = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B, C\}$$

$$V_T = \{a, b, c\}$$

$$P: S \Rightarrow AA/AC/CA/ACA/aAa/aa/bB/b/cC/c/\lambda$$

$$A \Rightarrow aAa/aa/bB/b/cC/c$$

$$B \Rightarrow bB/b/cC/c$$

$$C \Rightarrow cC/c$$

Elde edilir.

4.3.4. Yararsız Değişken, Simge ve Kurallar=

Eğer u simgelerden birisi, bu dilbilgisi tarafından türetilen tümce-lerin hiçbirinde yer almayorsa, bu u simgeye, yararsız simge denir.

Eğer değişkenlerden birisi, en az bir tümce türetilirken, elde edilen tümcesel yapıların en az birinde yer alıyorsa bu değişken yararlı bir değişkendir. Bu özelliği taşımayan

$S \Rightarrow \alpha_1 A \alpha_2 \Rightarrow u$ değişkenler ise yararsız değişken denir.

Eğer bir dilbilgisinin yeniden yazma kurallarından biri, en az bir tümcenin türetilmesi için kullanılıyorsa bu kural, yararlı kuraldır.

Bir A değişkeninin yararlı bir değişken olması için

1. Bu değişkenden başlanarak en az bir u simge dizisinin türetilmesi, (Alg 4.5)

$$A^* \Rightarrow u \quad u \in V_T^* \quad (u: \text{dilin bir tümcesi olması şart değil})$$

2. Başlangıç değişkeninden bu değişkene ulaşmanın mümkün olması (Alg 4.6)

$$S^* \Rightarrow \alpha_1 A \alpha_2 \text{ gereklidir.}$$

Algoritma 4.5 U_0 Simge dizisi türeten değişkenler kümesinin bulunması.

1. $T_D = \{A \mid (A \Rightarrow w) \in P \text{ ve } w \in V_T^*\}$;

2. ($T_D = T_{Eski}$) oluncaya kadar aşağıdaki işlemleri tekrarla: 2.1. $T_{Eski} = T_D$
2.2. Dilbilgisindeki her değişken (A) için aşağıdaki işlemleri tekrarla.

2.2.1. Her $(A \Rightarrow \beta)$ kuralı için aşağıdaki işlemleri tekrarla

Eğer $\beta \in (T_{Eski} \cup V_T)^*$ ise A'yı T_D 'ye ekle;

son.2.2.1.

son.2.2.

son.2.

(19)

Algoritma 4.6 S' den ulaşılabilen değişkenler kümesinin bulunması:

1. $T_u = \{S\}$;

2. $T_{Eski} = \emptyset$;

3. ($T_u = T_{Eski}$) oluncaya kadar aşağıdaki işlemleri tekrarla:

3.1. $T_{yeni} = T_u - T_{Eski}$;

3.1.1. $T_{Eski} = T_u$;

3.2. (T_{yeni})'de her değişken için:

3.2.1. Her $A \Rightarrow \beta$ için

β deki tüm değişkenleri T_u 'ya ekle.

son.3.2.1.

son.3.2.

son.3.

ÖRNEK 4.7: $G_{4.7} = \langle V_N, V_T, P, S \rangle$
 $V_N = \{S, A, B, C, D, E, F\}$
 $V_T = \{a, b\}$

P : $S \Rightarrow B \mid AC \mid BS$

$A \Rightarrow aA \mid aF$

$B \Rightarrow CF \mid b$

$C \Rightarrow cC \mid D$

$D \Rightarrow aD \mid C \mid BD$

$E \Rightarrow aA \mid BSA$

$F \Rightarrow bB \mid b$

$G_{4.7}$ ye eşdeğer yararız
değişken, simge ve kural içermeyen
bir dilbilgisi bulunmak için önce
 $G_{4.7}$ deki değişkenlerden hangisinin
 U_0 simge dizisi türeten değişkenler
kümesi olduğunu bulmak gerekir.
Alg. 4.5 kullanılarak $T_D = \{S, A, B, E, F\}$
bulunur. C ve D yararlı değilken
diğer değişkenler bu kümede yer almıyorlar.
 T_D yararlı değişkenleri (bulunmayan) içerir.

$G'_{4.7} = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B, E, F\}$

$V_T = \{a, b\}$

P : $S \Rightarrow B \mid BS$

$A \Rightarrow aA \mid aF$

$B \Rightarrow b$

$E \Rightarrow aA \mid BSA$

$F \Rightarrow bB \mid b$

C ve D atılınca gündeki db. elde edilir
2. adım hangi değişkenlere S' den ulaşılabilen
jüze bulunmaktadır. h.b. kullanılarak $T_u = \{S, B\}$
bulunur. A, E, F yararlı değilken atılır

$G = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, B\}$

$V_T = \{a, b\}$

P : $S \Rightarrow B \mid BS$

$B \Rightarrow b$

*NOT: Yararlı değilken
silinirken, bulunduğu
kuralda da silinir

4.5
 U_0 simge
türetenler
için
4.6

4.4. Normal Biçimler =

CFL (tür-2), dilbilgisi için Chomsky ve Greibach Normal Formları tanımlanmaktadır.

4.4.1. Chomsky Normal Form

Eğer, bağımlıdan bağımsız bir dilbilgisinin yeniden yazma kurallarının tümü

$$S \Rightarrow A$$

$$A \Rightarrow BC$$

$$A \Rightarrow \alpha \quad : \quad A, B, C \in V_N, \alpha \in V_T \text{ biçimindeyse CNF'dir}$$
$$BC \in V - \{S\}$$

Tür-2'ye CNF'e Dönüştürme : Eğer bir dilbilgisi CNF veya GNF'ye çevrilmel isteniyorsa o dilbilgisinin tür-2 olması şarttır. Tür-2 değilse CNF veya GNF ile ilişkilendirilemez.

~~B $\Rightarrow$ B~~ CFL değil. CNF veya GNF olamaz.

CNF'e uygun olmayan kurallar değiştirilmelidir. Uygun olan kurallar korunur ($A \Rightarrow BC$, $A \Rightarrow \alpha$), Diğer yeniden yazma kuralları atılır.

1. CNF'e uygun olmayan kuralın (K) sağ tarafının ilk simgesi bir değişken ya da bir ϵ simge olabilir.

1.1. Eğer ilk simge bir değişken ise :

$A \Rightarrow BX_2X_3...X_k$ $A, B \in V_N$ $X_2, X_3, ..., X_k \in V$ biçimindedir. Bu durumda değişkenler kümesine (D_1 gibi) yeni bir değişken eklenir

$$A \Rightarrow BD_1 \quad (K_1)$$

$$D_1 \Rightarrow X_2X_3...X_k \quad (K_2)$$

1.2. Eğer ilk simge bir ϵ simge ise :

$$A \Rightarrow \epsilon X_2X_3...X_k \quad A \in V_N \quad \epsilon \in V_T \quad X_2, X_3, ..., X_k \in V$$

$$A \Rightarrow C_1D_1 \quad (K_1) \text{ eklenir.}$$

$$D_1 \Rightarrow X_2X_3...X_k \quad (K_2)$$

$$C_1 \Rightarrow \epsilon \quad (K_3)$$

2. Eğer K yarı kurallardan bir CNF değilse (K_1) aynı işlemler ve tüm diğer uygun kur. izlenir.

ÖRNEK

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aB \quad (1)$$

$$S \Rightarrow bA \quad (2)$$

$$A \Rightarrow aS \quad (3)$$

$$A \Rightarrow bAA \quad (4)$$

$$\bullet A \Rightarrow a \quad (5)$$

$$B \Rightarrow bS \quad (6)$$

$$B \Rightarrow aBB \quad (7)$$

$$\bullet B \Rightarrow b \quad (8)$$

5 ve 8 CNF'ı sağla

$$S \Rightarrow a$$

$$A \Rightarrow BC \quad BC \in V_N^+$$

$$A \Rightarrow a$$

$$(1) S \Rightarrow aB$$

$$S \Rightarrow CaB \quad \vee \quad Ca = a$$

$$(2) S \Rightarrow bA$$

$$S \Rightarrow CbA \quad \vee \quad Cb = b$$

$$(3) A \Rightarrow aS$$

$$A \Rightarrow CaS$$

$$(4) A \Rightarrow bAA$$

$$A \Rightarrow CbD_1 \quad \vee \quad D_1 \Rightarrow AA$$

$$(6) B \Rightarrow bS$$

$$B \Rightarrow CbS$$

$$(7) B \Rightarrow aBB$$

$$B \Rightarrow CaD_2 \quad \vee \quad D_2 \Rightarrow BB$$

CNF hâline $G = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow C_a B | C_b A$$

$$A \Rightarrow C_a S | C_b D_1 | a$$

$$B \Rightarrow C_b S | C_a D_2 | b$$

$$C_a \Rightarrow a$$

$$C_b \Rightarrow b$$

$$D_1 \Rightarrow AA$$

$$D_2 \Rightarrow BB$$

(20)

37

4.4.2. Greibach Normal Form

$$S \Rightarrow a$$

$$A \Rightarrow a\alpha \quad : \quad A \in V_N, \alpha \in V_T, \alpha \in V_N^*$$

Türetme kurallarının hepsi yandaki gibi olan dilbilgisi CNF'dadır.

Bağılandıran bağımsız bir dilbilgisi CNF'e dönüştürürken bu özellik üzerinde durulmalıdır (Lemma).

Lemma 4.1. = Bağılandıran bağımsız dilbilgisinin yandaki yazma kurallarından birisi

$A \Rightarrow \alpha_1 B \alpha_2 (k_1)$ olsun. Tüm B kuralları $B \Rightarrow \beta_1 | \beta_2 | \beta_3 \dots | \beta_n$ ise bu dilbilgisinden k_1 kural çıkarılıp yerine

$$A \Rightarrow \alpha_1 \beta_1 \alpha_2 | \alpha_1 \beta_2 \alpha_2 | \dots | \alpha_1 \beta_n \alpha_2 \quad \text{kuralları konulduğunda eşdeğer dilbilgisi elde edilir}$$

Lemma 4.2. Eğer CFL doğrudan özüneli türetme kuralları içeriyorsa bu kurallar kaldırılabilir.

Dilbilgisinin A kurallarının bir kısmını doğrudan özüneli ise $A \Rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_r (j_1)$ gruplara ayırır. ve $A \Rightarrow \beta_i B \quad (i=1,2,\dots,r)$

$$A \Rightarrow \beta_1 | \beta_2 | \dots | \beta_r \quad (j_2)$$

$$B \Rightarrow \alpha_j \quad j=1,2,\dots,r$$

$$B \Rightarrow \alpha_j B \quad j=1,2,\dots,r$$

CFL'ye GNF'ye Dönüştürme

Bir CNF verildiğinde aşağıdaki adımlar kullanılarak GNF'ye dönüştürülebilir. GNF'ye dönüştürme için $CFL \Rightarrow CNF \Rightarrow GNF$ sırasıyla uygulanır.

1. Adım

1.1. Sözdüm değişkenler $A_1, A_2, \dots, A_L$ gibi dizimli değişkenlerle değiştirilir. S 'nin yerine A_1 yazılır.

1.2. Lemma 4.1 kullanılarak

$A_i \Rightarrow A_j \gamma$ $\gamma \neq \epsilon$ koşuluyla sağlanacak biçime dönüştürülür.

2. Adım

2.1. Lemma 4.2 kullanılarak doğrudan üretilen kuralların yerine yeni kurallar konur. $B_1, B_2, \dots$ gibi yeni değişkenler eklenir.

3. Adım

3.1. Lemma 4.1 ile $A_{L-1}, A_{L-2}, \dots, A_2, A_1$ sırasındaki tüm A_i kuralları GNF'ye dönüştürülür.

3.2. Lemma 4.1 kullanılarak B_j kuralları GNF'ye dönüştürülür.
(yeni eklenen 2.4.2 kuralları)

ÖRNEK

$G = \langle V_N, V_T, P, S \rangle$

$V_N = \{A_1, A_2, A_3\}$

$V_T = \{a, b\}$

$P: A_1 \Rightarrow A_2 A_3 \quad (1)$

$A_2 \Rightarrow A_3 A_1 \quad (2)$

$A_2 \Rightarrow b \quad (3)$

$A_3 \Rightarrow A_1 A_2 \quad (4)$

$A_3 \Rightarrow a \quad (5)$

Bu dilbilgisi CNF dir. Fakat 1, 2 ve 4 GNF'ye dönüştürülebilir.

1. $A_i \Rightarrow A_j \gamma$ $\gamma \neq \epsilon$ $\rightarrow$ (koşulu sağlanacak) şekilde kurallar düzenlenir.
1 ve 2 bu kuralı sağlıyor 4 nolu kural.

$A_3 \Rightarrow A_1 A_2$ yerine $A_3 \Rightarrow A_2 A_3 A_1$ yazılır.
Yeni Kural da sağlanmadı.

$A_3 \Rightarrow A_2 A_3 A_1$ yerine $\left\{ \begin{array}{l} A_3 \Rightarrow A_2 A_1 A_3 A_2 \\ A_3 \Rightarrow b A_3 A_2 \end{array} \right\}$ konular.

Bu durumda yeni kurallar. $\rightarrow$ 2. Üretilen Kuralların 2.4.2 ile yok edilmesi.

$A_1 \Rightarrow A_2 A_3$

$A_2 \Rightarrow A_3 A_1$

$A_2 \Rightarrow b$

$A_3 \Rightarrow A_2 A_1 A_3 A_2$

$A_3 \Rightarrow b A_3 A_2$

$A_3 \Rightarrow a$

Sadece $A_3 \Rightarrow A_2 A_1 A_3 A_2$ doğrudan üretilir. Bu durumda.

$\rightarrow A_3 \Rightarrow A_2 A_1 A_3 A_2$ yerine $\left\{ \begin{array}{l} A_3 \Rightarrow b A_3 A_2 B_3 \\ A_3 \Rightarrow a B_3 \\ B_3 \Rightarrow A_1 A_3 A_2 \\ B_3 \Rightarrow A_1 A_3 A_2 B_3 \end{array} \right\}$ A_3
 B_3 : yeni değişken.

Bu durumda olusan yeni kurallar:

$$A_1 \Rightarrow A_2 A_3$$

$$A_2 \Rightarrow A_3 A_1$$

$$A_1 \Rightarrow b$$

$$A_3 \Rightarrow A_2 A_1 A_3 A_2$$

$$A_2 \Rightarrow b A_3 A_2$$

$$A_3 \Rightarrow a$$

$$A_3 \Rightarrow b A_3 A_2 B_3$$

$$A_3 \Rightarrow a B_3$$

$$B_3 \Rightarrow A_1 A_3 A_2$$

$$B_3 \Rightarrow A_1 A_3 A_2 B_3$$

(21)

3.1. de $A_2 \Rightarrow A_3 A_1$ de sadece A_3 uyar
ilk gosterilen dog. dogrultulur ve A_{L-1} 'den
baslanir.

3.1. Bu adimda Lemma 4.1. kullanilacak A_2, A_1 ve B_3 GNF'ye uygun hale getirilir.
En yuksek indis 3 A_3 A_2 den baslanir.

$$\left\{ \begin{array}{l} A_2 \Rightarrow A_3 A_1 \\ A_2 \Rightarrow b \end{array} \right\} \text{ yeni} \quad \left\{ \begin{array}{l} A_2 \Rightarrow b A_3 A_2 A_1 \\ A_2 \Rightarrow a A_1 \\ A_2 \Rightarrow b A_3 A_2 B_3 A_1 \\ A_2 \Rightarrow a B_3 A_1 \\ A_2 \Rightarrow b \end{array} \right\} \text{ konulur. } A_2$$

$$\begin{array}{l} A_1 \Rightarrow A_2 A_3 \text{ yeni} \\ \downarrow \\ \text{yeni } A_2 \text{ dog. kullanilir.} \end{array} \quad \left\{ \begin{array}{l} A_1 \Rightarrow b A_3 A_2 A_1 A_3 \\ A_1 \Rightarrow a A_1 A_3 \\ A_1 \Rightarrow b A_3 A_2 B_3 A_1 A_3 \\ A_1 \Rightarrow a B_3 A_1 A_3 \\ A_1 \Rightarrow b A_3 \end{array} \right\} \text{ konulur } A_1$$

$$\begin{array}{l} B_3 \Rightarrow A_1 A_3 A_2 \\ \downarrow \\ \text{yeni } A_1 \text{ in} \\ \text{kullanilir} \end{array} \quad \left\{ \begin{array}{l} B_3 \Rightarrow b A_3 A_2 A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow a A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow b A_3 A_2 B_3 A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow a B_3 A_1 A_3 A_3 A_2 \\ B_3 \Rightarrow b A_3 A_3 A_2 \end{array} \right\} B_3$$

$$B_3 \Rightarrow A_1 A_3 A_2 B_3 \text{ yeni} \quad \left\{ \begin{array}{l} B_3 \Rightarrow b A_3 A_2 A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow a A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow b A_3 A_2 B_3 A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow a B_3 A_1 A_3 A_3 A_2 B_3 \\ B_3 \Rightarrow b A_3 A_3 A_2 B_3 \end{array} \right\} \text{ ve } B_3 \text{ silinir} \\ \text{12 kural}$$

bitki. $G = \langle V_N, V_T, P, S \rangle$
seklinde yazilir
ve b.t.r.

Anlık Tanımlar

- PDA'nın belirli bir andan sonraki davranışlarını kestirebilmek için o ana ilişkin üç bilginin bilinmesi gerekir.
- PDA'nın (sonrakı den. bir). bulunduğu durum.
 - Giriş dizisinin henüz işlenmemiş kısmı.
 - Yığılma içeriği.

$$(ID) = (p, v, X)$$

p: PDA'nın durumu

v: giriş dizisine işlenmemiş kısmı

X: yığılma içeriği

PDA'nın tanıdığı Dil.

Dizileri tanıma açısından PDA'nın iki alt modeli vardır. Bu modeller:

- Uç durumlara tanıyan PDA
- Boş yığılma tanıyan PDA

Uç durumlara tanıyan PDA modelinde dizinin tüm simgeleri okunduktan sonra, yığıt içeriğine bakılmaz ve, PDA bir uç durumda bulunuyorsa bu dizi PDA tarafından tanınır.

$$T(M) = \{ w \mid w \in V_T^*, (q_0, w, z_0) \xrightarrow{*} (p, \lambda, \gamma), p \in F \}$$

Boş yığılma tanıyan PDA modelinde ise tüm simgeler okunduktan sonra yığıt boşalmıyorsa dizi PDA tarafından tanınır. Bu modelde son duruma gelirse bile yığıt boş değilse dizi tanınmaz.

$$T(M) = \{ w \mid w \in V_T^*, (q_0, w, z_0) \xrightarrow{*} (p, \lambda, \lambda), p \in Q \}$$

ÖRNEK: L_{5.1} dilini aşağıdaki gibi tanımlanıyor:

$$L_{5.1} = \{ w c w^R \mid w \in (0+1)^* \} \quad \text{Bu dil türeten dilbilgisi:}$$

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{ S \}$$

$$V_T = \{ 0, 1, c \}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid c$$

$$L_{5.1} \text{ i tanıyan PDA } M = \langle Q, \Sigma, \Gamma, \delta, q_0, z_0, \emptyset \rangle$$

$$Q = \{ q_0, q_1 \}$$

$$\Sigma = \{ 0, 1, c \}$$

$$\Gamma = \{ 0, 1, z_0 \}$$

PDA'nın kabul ettiği dil: Bu PDA'nın tanıdığı dizgiler $\{0,1\}$ alfabesindeki herhangi bir w altdizisi ile bunun tersinden oluşmaktadır w ile w^R arasında bir 'c' simgesi yer almaktadır. Bu dizileri tanıyan PDA

* Önce w 'nin simgeleri okunur, okunan her simge yığılta eklenir. Ekleme sırasında PDA q_0 durumunda bulunur.

* c'yi okudugunda w 'nin bittiğini anlar, yığıtta bir değişiklik yapmaz ve q_1 durumuna geçer.

* Sonra w^R 'nin simgelerini okur ve okuduğu her simge için eğer yığıt tepesinde aynı simge varsa bu simgeyi yığıttan siler (push).

* Eğer dizinin tüm simgeleri okunduktan sonra yığıt tepesinde z_0 varsa bunu da siler ve yığıtı tamamen boşaltır.

$$\begin{aligned} \delta: \delta(q_0, 0, z_0) &= (q_0, 0z_0) \\ \delta(q_0, 1, z_0) &= (q_0, 1z_0) \\ \delta(q_0, c, z_0) &= (q_1, z_0) \\ \delta(q_0, 0, 0) &= (q_0, 00) \\ \delta(q_0, 1, 0) &= (q_0, 10) \\ \delta(q_0, 0, 1) &= (q_0, 01) \\ \delta(q_0, 1, 1) &= (q_0, 11) \\ \delta(q_0, c, 0) &= (q_1, 0) \\ \delta(q_0, c, 1) &= (q_1, 1) \\ \delta(q_1, 0, 0) &= (q_1, 2) \\ \delta(q_1, 1, 1) &= (q_1, 2) \\ \delta(q_1, 2, z_0) &= (q_1, 2) \end{aligned}$$

Gecişler birbirinden bağımsızdır ve olasılıkla tüm geçişlerdir.

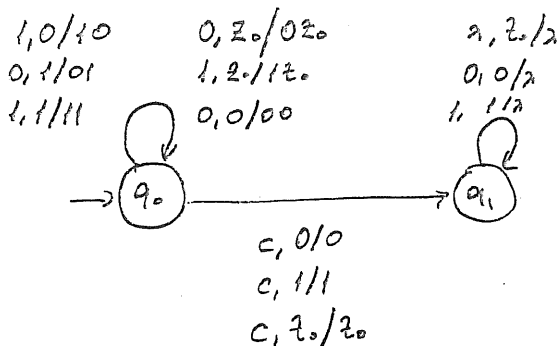
→ POP silinen

0, 1/11

→ push edilen.

↓
read
simgeyi
okunmuş
gibi yaz

(23)



ÖRNEK

$L = \{ ww^R \mid w \in (0+1)^* \}$ bu dilin türetilebilirliği

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{ S \}$$

$$V_T = \{ 0, 1 \}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid \lambda$$

Bu dilin tanıyan PDA

$$M_{5.2} = \langle Q, \Sigma, \Gamma, \delta, q_0, z_0, \Phi \rangle$$

$$Q = \{ q_0, q_1 \}$$

$$\Sigma = \{ 0, 1 \}$$

$$\Gamma = \{ A, B, Z_0 \}$$

PDA'nın kabul ettiği dil = PDA'nın tanıdığı dil, $\{ 0, 1 \}$ alfabesindeki herhangi bir w alt dizisi ile bunun tersinden okunmaktadır. w ile w^R arasında herhangi bir simge yer almamaktadır. Bu dilin tanıyan PDA

• Önce w 'nin simgeleri okunur. Yığıta her 0 için A, her 1 için de bir B eklenir. PDA'nın yığıt alfabesi giriş alfabesindeki simgeleri içerebilir, fakat bu simgeleri içermeyebilir den

• w 'nin simgeleri okunurken PDA q_0 durumundadır. w bitip w^R başladığında PDA q_1 durumuna geçer. Ancak giriş dizisinde w 'nin bitip w^R 'nin başladığını belirten özel bir simge yoktur. Okunan simge bir önceki simgenin aynı ise bu simge w 'nin bir sonraki simgesine eklenebilir, diğer w^R 'nin ilk simgesi de olabilir. Bu koşullara da ilk hareket tanımlanmalıdır. 1. hareket PDA eklem durumunda kalır ve yığıta eklem yapar; ikinci hareket ise PDA silme durumuna geçer ve yığıtın tepesindeki simgeyi siler.

• Silme durumuna geçen PDA w^R 'nin simgelerini okur ve okuduğu her simge için eğer yığıt tepesinde bu simge varsa bunu yığıttan siler. (0 için A, 1 için B silinir).

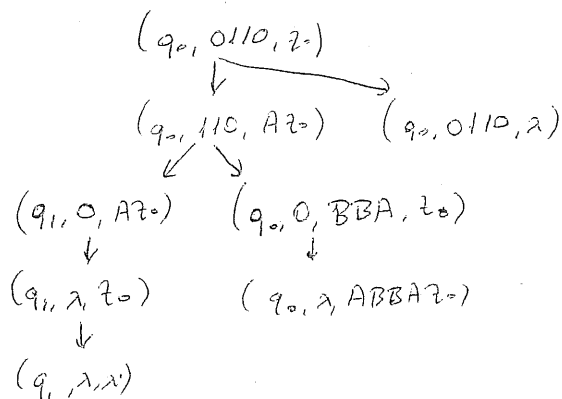
• Giriş dizisinin tüm simgeleri okunduktan sonra eğer yığıt tepesinde Z_0 varsa PDA bunu da siler ve yığıt tamamen boşalır.

Yukarıdaki sözcük olarak tanımlanan işlevleri yapan PDA'nın herdet işlevi birimisel olarak tanımlanabilir.

$$\begin{aligned}
 \delta: \quad & \delta(q_0, 0, z_0) = (q_0, Az_0) \\
 & \delta(q_0, 1, z_0) = (q_0, Bz_0) \\
 & \delta(q_0, \lambda, z_0) = (q_1, \lambda) \\
 & \delta(q_0, 0, A) = \{(q_0, AA), (q_1, \lambda)\} \\
 & \delta(q_0, 1, A) = (q_0, BA) \\
 & \delta(q_0, 0, B) = (q_0, AB) \\
 & \delta(q_0, 1, B) = \{(q_0, BB), (q_1, \lambda)\} \\
 & \delta(q_1, 0, A) = (q_1, \lambda) \\
 & \delta(q_1, 1, B) = (q_1, \lambda) \\
 & \delta(q_1, \lambda, z_0) = (q_1, \lambda)
 \end{aligned}$$

(24)

$w = 0110$ dizisini bu PDA tarafından tanımlanır.



(q_1, λ, λ) ya ulaşılır.
Bu makine 0110 dizisini tanımlar.

0,11 alfabeesinde palindromları tanıyan bir dil vardır. $\{ \lambda, 0, 1, 010, 111, 11, 00, 001100 \}$

$$L = \{ w w^R + w 0 w^R + w 1 w^R \mid w \in \{0,1\}^* \}$$

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{ S \}$$

$$V_T = \{ 0, 1 \}$$

$$P: S \Rightarrow 0S0 \mid 1S1 \mid 011 \mid 1$$

Bu dili tanıyan PDA

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, z_0, \phi \rangle$$

$$Q = \{ q_0, q_1 \}$$

$$\Sigma = \{ 0, 1 \}$$

$$\Gamma = \{ 0, 1, z_0 \}$$

PDA'nın çalışma şekli. PDA'nın tanıdığı diziler $\{0,1\}$ alfabesindeki her

hangi bir w alt dizisi ile bunun tersinden oluşmaktadır w ile w^R arasında 0,1 geçişleri veya birbirine geçiş.

* w bitip w^R başladığında PDA q_1 durumuna geçecektir. Fakat w 'nin bitip w^R 'nin başladığını sistem ayrıca bir işleme yoktur okunan işleme w 'nin son işleme, w^R 'nin ilk işleme veya aradaki işleme (w/c) olabilir PDA deterministik değildir

* Silme Durumuna geçiş PDA w^R 'nin işaretlerini okur ve okuduktan ^{işleme} yığıtın en tepesinde varsa bunu yığıttan çıkarır.

* Giriş dizisinin tüm işaretleri okunduktan sonra, eğer yığıtın tepesinde 0 varsa PDA bunu da siler ve yığıt tamamen boşalır.

Yukarıdaki işlemleri yapan PDA'nın geçiş fonksiyonları aşağıdadır

$$\begin{aligned}\delta: \delta(q_0, 0, z_0) &= \{(q_0, 0z_0), (q_1, z_0)\} \\ \delta(q_0, 1, z_0) &= \{(q_0, 1z_0), (q_1, z_0)\} \\ \delta(q_0, \lambda, z_0) &= (q_1, \lambda) \\ \delta(q_0, 0, 0) &= \{(q_0, 00), (q_1, 0), (q_1, \lambda)\} \\ \delta(q_0, 1, 0) &= \{(q_0, 10), (q_1, 0)\}\end{aligned}$$

$$\begin{aligned}\delta(q_0, 0, 1) &= \{(q_0, 01), (q_1, 1)\} \\ \delta(q_0, 1, 1) &= \{(q_0, 11), (q_1, 1), (q_1, \lambda)\}\end{aligned}$$

$$\begin{aligned}\delta(q_1, 0, 0) &= (q_1, \lambda) \\ \delta(q_1, 1, 1) &= (q_1, \lambda) \\ \delta(q_1, \lambda, z_0) &= (q_1, \lambda)\end{aligned}$$

ÖRNEK $L = \{a^i b^j a^k \mid i, j, k, n \geq 1\}$

Bu dil türetken dilidir.

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow ABA$$

$$A \Rightarrow aAa$$

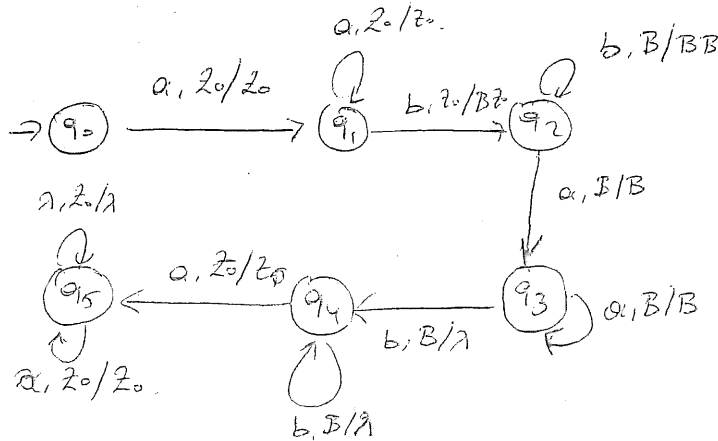
$$B \Rightarrow bBb \mid bAb$$

PDA'nın Çalışma İlkesi = PDA'nın tanıdığı diller, 5 öbektir olmaktadır.
Her öbekte en az bir simge yer almaktadır.

1, 3 ve 5. öbektelerdeki simgeler a, 2 ve 4. öbektelerdeki simgeler b'dir.
PDA, a'ların en az 1 adet olduğunu ve b'lerin sayısının eşit olduğunu denetlemek için kullanılır. Bunun için yığıt b'lerin eşit sayısını bulmak için kullanılır. Bunun için 2. öbekte b'ler yığılabilirken 4. öbekte b'ler yığılmağa çekilir.

$$\begin{aligned} \delta: \delta(q_0, a, z_0) &= (q_1, z_0) \\ \delta(q_1, a, z_0) &= (q_1, z_0) \\ \delta(q_1, b, z_0) &= (q_2, Bz_0) \\ \delta(q_2, b, B) &= (q_2, BB) \\ \delta(q_2, a, B) &= (q_3, B) \\ \delta(q_3, a, B) &= (q_3, B) \\ \delta(q_3, b, B) &= (q_4, \lambda) \\ \delta(q_4, b, B) &= (q_4, \lambda) \\ \delta(q_4, a, z_0) &= (q_5, z_0) \\ \delta(q_5, a, z_0) &= (q_5, z_0) \\ \delta(q_5, \lambda, z_0) &= (q_5, \lambda) \end{aligned}$$

(25)



=PDA'nın Deterministik Olma Kuralı =

Her CFL bir PDA bulunur. Fakat bulunan her PDA deterministik değildir. Deterministik bir PDA:

- Her $\delta(q, a, X)$ için tanımlı en çok bir hareket olmalıdır.
- $\delta(q, \lambda, X)$ için bir hareket tanımlı ise, hiçbir giriş simgesinin $\delta(q, a, X)$ hareketinin tanımlı olmadığı ~~gerekir~~ gerekir.

Yukarıdaki makinede q_0, q_1, q_2, q_3 ve q_4 için det. Anlaşır. q_5 için tanımlı hareketler det. değildir. Bu PDA det. değildir.

5.2. Bağımlıdan Bağımsız (CFG) Dilbilgisi ve PDA Eşdeğerliği

Bir CFG'ye eşdeğer bir PDA varsa, bu ~~PDA~~ CFG'ye eşdeğer CFG'de vardır.

5.2.1. Verilen Bir CFG'nin Eşdeğer PDA'sını Bulma.

* Eğer G bağımlıdan bağımsız bir dilbilgisi ise bu dilbilgisinin türettiği bağımlıdan bağımsız dil $L(G)$ tanıyan bir PDA vardır.

CFG G , PDA'ya M ile gösterilmek verilen bir CFG için $T(M) = L(G)$ olan PDA iki yolla bulunur.

CFG Eşdeğer PDA'ya bulma için 1. yöntem. (Bos yığıtlı tanıyan PDA)

Bağımlıdan bağımsız bir dilbilgisi $G = \langle V_N, V_T, P, S \rangle$ dir.

Bu yöntem için verilen dilbilgisinin Grebach Normal Formda olması gerekir.

$$A \Rightarrow \alpha$$

$$A \Rightarrow \alpha : A \in V_N, \alpha \in V_T, \alpha \in V_N^*$$

$$M = \langle Q, \Sigma, \Gamma, q_0, \delta, z_0, F \rangle$$

$$Q = \{q_0\}$$

$$\Sigma = V_T$$

$$\Gamma = V_N$$

$$z_0 = S$$

$$F = \emptyset$$

$$\delta : A \Rightarrow \alpha \text{ için } \delta(q_0, \alpha, A) = (q_0, \alpha) \text{ hareketi.}$$

$$\alpha = \lambda \text{ ise } A \Rightarrow a \text{ için } \delta(q_0, a, A) = (q_0, \lambda) \text{ hareketi: boş.}$$

$F = \emptyset$ olduğuna göre, boş yığıtlı tanıyan bir PDA oluşturulmuş.

PDA'nın

* Giriş alfabesi, dilbilgisinin α stringleri kümesine

* Yığıt alfabesi " " satabilim değişkenler "

* Yığıt başl. değişkeni " " başl. değişkeni olan S 'e

* Durumlar kümesi tek durum olan $\{q_0\}$

* Durum geçişleri, dilbilgisi tarafından yeniden türetilen kuralardan

ÖRNEK: $G = \langle V_N, V_T, P, S \rangle$

$$V_N = \{S\}$$

$$V_T = \{+, -, *, /, v, c\}$$

$$P: S \Rightarrow +SS | -SS | *SS | /SS | v/c$$

Bu dilbilgisiyle türetilebilir olan PDA

$$M = \langle Q, \Sigma, \Gamma, \delta, F, z_0, q_0 \rangle$$

(26)

$$Q = \{q_0\}$$

$$\Sigma = \{+, -, *, /, v, c\}$$

$$\Gamma = \{S\}$$

$$F = \emptyset$$

$$z_0 = S$$

$$\delta: \begin{aligned} \delta(q_0, +, S) &= (q_0, SS) \\ \delta(q_0, -, S) &= (q_0, SS) \\ \delta(q_0, *, S) &= (q_0, SS) \\ \delta(q_0, /, S) &= (q_0, SS) \\ \delta(q_0, v, S) &= (q_0, \lambda) \\ \delta(q_0, c, S) &= (q_0, \lambda) \end{aligned}$$

Dilbilgisiyle türetilebilir olan dizi.

$$\begin{aligned} S &\Rightarrow *SS \Rightarrow *+SSS \Rightarrow *+vSS \Rightarrow *+v/SSS \Rightarrow *+v/vSS \Rightarrow *+v/vcS \Rightarrow *+v/vc-SS \\ &\Rightarrow *+v/vc-SS \Rightarrow *+v/vc-vS \Rightarrow *+v/vc-v*SS \Rightarrow *+v/vc-v*cS \Rightarrow *+v/vc-v*c v \\ w &= (v+v/c)^*(v-cv) \end{aligned}$$

$$\begin{aligned} (q_0, *+v/vc-v*c v, S) &\vdash (q_0, +v/vc-v*c v, SS) \\ \vdash (q_0, v/vc-v*c v, SSS) &\vdash (q_0, /vc-v*c v, SS) \vdash (q_0, vc-v*c v, SSS) \\ \vdash (q_0, vc-v*c v, SS) &\vdash (q_0, -v*c v, S) \vdash (q_0, v*c v, S) \vdash (q_0, *cv, S) \\ \vdash (q_0, cv, SS) &\vdash (q_0, v, S) \vdash (q_0, \lambda, \lambda) \end{aligned}$$

Yığılta boş dizi: ikinci PDA w'yi tanımlar

ÖRNEK: $\{a, b\}$ alfabeisinde eşit sayda a ve b içeren CFG.

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S, A, B\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow bA|aB$$

$$A \Rightarrow bAA|a|a$$

$$B \Rightarrow aBB|bS|b$$

$$M = \langle Q, \Sigma, \Gamma, \delta, F, q_0, z_0 \rangle$$

$$Q = \{q, \lambda\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{S, A, B\}$$

$$z_0 = S$$

$$F = \emptyset$$

$$\delta: \delta(q_0, b, S) = (q_0, A)$$

$$\delta(q_0, a, S) = (q_0, B)$$

$$\delta(q_0, b, A) = (q_0, AA)$$

$$\delta(q_0, a, A) = \{(q_0, S), (q_0, \lambda)\}$$

$$\delta(q_0, a, B) = (q_0, BB)$$

$$\delta(q_0, b, B) = \{(q_0, S), (q_0, \lambda)\}$$

Deterministik değildir.

CFG'nin Eşdeğeri PDA'nın Bulunması. (Uç Durumda Tanıyan PDA)



Bu modelde, a : giriş sendikden okunan simge

z : yığılın tepesindeki simgeyi

α : yığılın altına okunan simgeyi değiştirir gösterir

Temel modelde a, α yerine λ gelir. Bu modelde z yerine de λ gelebilir.

$$\delta(q_i, a, \lambda) = (q_j, \alpha) \quad \text{yerine} \quad \forall k \in \Gamma \quad \delta(q_i, a, z_k) = (q_j, \alpha, z_k)$$

$$G = \langle V_N, V_T, P, S \rangle$$

(27)

$$M = \langle Q, \Sigma, \Gamma, q_0, \delta, z_0 \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = V_T$$

$$\Gamma = V_N \cup V_T \cup \{z_0\}$$

$$F = \{q_2\}$$

$$\delta: \quad \delta(q_0, \lambda, \lambda) = (q_1, \lambda) \quad \text{ve} \quad \delta(q_1, \lambda, z_0) = (q_2, \lambda) : \text{Dışarıdan başlanıyor}$$

okunak her PDA için tanımlanır

$$\delta(q_1, a, a) = (q_1, \lambda)$$

$$\text{her } A \Rightarrow \alpha \text{ için PDA'da } \delta(q_1, \lambda, A) = (q_1, \alpha) \text{ tanımlanır}$$

Okuyan PDA uç durumda tanır yani sonunda yığılın da boşaltılır

Örnek: $\{a, b\}$ alfabetinde

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_N = \{S\}$$

$$V_T = \{a, b\}$$

$$P: S \Rightarrow aSb \mid aSbb \mid ab \mid abb$$

$$L = \{a^n b^n \mid 1 \leq n, n \leq m \leq 2n\}$$

Verilen dilbilgisiye eşdeğer PDA:

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, z_0, F \rangle$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, S, Z\}$$

$$F = \{q_2\}$$

$$\delta: \delta(q_0, \lambda, \lambda) = (q_1, S)$$

$$\delta(q_1, a, a) = (q_1, \lambda)$$

$$\delta(q_1, b, b) = (q_1, \lambda)$$

$$\delta(q_1, \lambda, S) = (q_1, aSb)$$

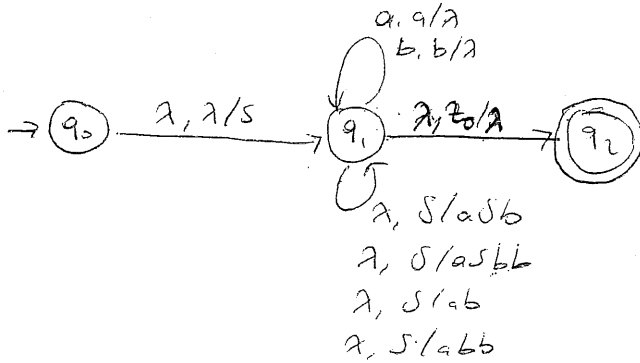
$$\delta(q_1, \lambda, S) = (q_1, aSbb)$$

$$\delta(q_1, \lambda, S) = (q_1, ab)$$

$$\delta(q_1, \lambda, S) = (q_1, abbb)$$

$$\delta(q_1, \lambda, Z) = (q_2, \lambda)$$

PDA Non-det. tir



5.2.2. Verilen bir PDA dan Eşdeğer CFG bulunması

* Eğer M bir PDA ise, türettiği dil bu PDA'nın $G(L) = T(M)$ bağlamdan-bağımsız bir dilbilgisi (CFG) vardır.

$$PDA: M = \langle Q, \Sigma, \Gamma, q_0, \delta, z_0, F \rangle$$

Verilen PDA'ya eşdeğer, Greibach bağlamdan-bağımsız dil

$$G = \langle V_N, V_T, P, S \rangle$$

$$V_T = \Sigma$$

$$V_N = \{ S, [q, A, p] \mid A \in \Gamma, p, q \in Q \}$$

P: PDA'nın her durumuna karşılık CFG'de

$$\forall q \in Q: S \rightarrow [q_0, z, q]$$

PDA'nın her hareketine karşılık

$$\forall \delta(q_i, a, A) = (q_j, B_1 B_2 \dots B_k) \in P :$$

Her $p_1, p_2 \dots p_k \in Q$ birleşim için bir yenden yazılabilir.

$$[q_i, A, p_k] \Rightarrow a [q_j, B_1, p_1] [p_1, B_2, p_2] \dots [p_{k-1}, B_k, p_k]$$

Bu yöntemle girilen verilerden bir PDA ya eşdeğer CFG. (28)

1. Dilbilgisinin uyarı simgeler kümesi, PDA'nın girilen alfabeti: $V_T = \Sigma$

2. Dilbilgisinin sınırlı değişkenleri, S de $[durum, yığıt simgesi, durum]$

uygulardan oluşur. Bu durumda PDA'da n adet durumda

k adet yığıt simgesi varsa CFG de $k^2 + 1$ adet sınırlı

değişken bulunacaktır.

Örneğin PDA'da durumlar kümesi $\{q_0, q_1\}$ yığıt alfabeti $\{A, B, Z\}$ ise $3(2)^2 + 1 = 13$ adet sınırlı değişken olacaktır.

$$V_N = \{S, [q_0, A, q_0], [q_0, A, q_1], [q_0, B, q_0], [q_0, B, q_1], [q_0, Z, q_0], [q_0, Z, q_1], [q_1, A, q_0], [q_1, A, q_1], [q_1, B, q_0], [q_1, B, q_1], [q_1, Z, q_0], [q_1, Z, q_1]\}$$

3. CFG'nin yenden yazılabilir kuralları aşağıdaki gibi bulunur.

3.1. $\forall q \in Q : S \Rightarrow [q_0, Z, q]$

3.2. PDA'nın $\delta(q_0, \lambda, A) = (q_1, \lambda)$ ya karşılık

$$[q_0, A, q_1] \Rightarrow \lambda$$

3.3. PDA'nın $\delta(q_0, a, A) = (q_1, a)$ ya karşılık

$$[q_0, A, q_1] \Rightarrow a$$

3.4. PDA'nın $\delta(q_0, a, A) = (q_1, B_1)$

$$[q_0, A, p_1] \Rightarrow a [q_1, B_1, p_1]$$

3.5. PDA'nın $\delta(q_0, a, A) = (q_1, B_1, B_2)$

$$[q_0, A, p_1] \Rightarrow a [q_1, B_1, p_1] [p_1, B_2, q_1]$$

~~3.6 PDA'nın~~

~~$$[q_0, A, p_1] \Rightarrow a(q_1, B_1, p_2)[p_1, B_2, p_2]$$~~

3.6. PDA'nın $\delta(q_0, a, A) = (q_1, B_1 B_2 B_3) \quad \forall p_1, p_2, p_3 \in Q$

$$[q_0, A, p_3] \Rightarrow a(q_1, B_1, p_2)[p_1, B_2, p_2][p_2, B_3, p_3]$$

ÖRNEK. Aşağıdaki PDA veriliyor.

$$M = \langle Q, \Sigma, \Gamma, q_0, \delta, z_0, \Phi \rangle$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{z_0, x\}$$

$$\delta: \delta(q_0, 0, z_0) = (q_0, x z_0)$$

$$\delta(q_0, 0, x) = (q_0, x x)$$

$$\delta(q_0, 1, x) = (q_1, \lambda)$$

$$\delta(q_1, 1, x) = (q_1, \lambda)$$

$$\delta(q_1, x, z) = (q_1, \lambda)$$

Tandem dil $L = \{a^n b^n \mid n \geq 1\}$

Bunu türeten CFG.

$$G = \langle V_N, V_T, P, S \rangle$$

$$z = x \quad \downarrow q_0, q_1$$

$$k = 2$$

$$n = 2$$

$$kn^2 + 1 = 2(2)^2 + 1 = 9..$$

$$V_T = \{0, 1\}$$

$$V_N = \{S, A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8\}$$

$$A_1 = [q_0, x, q_0]$$

$$A_2 = [q_0, x, q_1]$$

$$A_3 = [q_0, z_0, q_0]$$

$$A_4 = [q_0, z_0, q_1]$$

$$A_5 = [q_1, x, q_0]$$

$$A_6 = [q_1, x, q_1]$$

$$A_7 = [q_1, z_0, q_0]$$

$$A_8 = [q_1, z_0, q_1]$$

Genel Grammar Kuralları.

• PDA'nın durumlarına karşılık oluşturulan kuralları:

$$S \Rightarrow [q_0, z_0, q_0]$$

$$S \Rightarrow [q_0, z_0, q_1]$$

$$\bullet \delta(q_0, 0, z_0) = (q_0, xz_0) \text{ horizontale Kette.}$$

$$\begin{aligned} [q_0, t_0, q_1] &\Rightarrow O[q_0, x, q_0][q_0, z_0, q_0] \\ [q_0, z_0, q_1] &\Rightarrow O[q_0, x, q_0][q_0, z_0, q_1] \\ [q_0, z_0, q_0] &\Rightarrow O[q_0, x, q_1][q_1, z_0, q_0] \\ [q_0, z_0, q_1] &\Rightarrow O[q_0, x, q_1][q_1, z_0, q_1] \end{aligned}$$

$$\bullet \delta(q_0, 0, x) = (q_0, xx) \text{ hor. Kette.}$$

$$\begin{aligned} [q_0, x, q_0] &\Rightarrow O[q_0, x, q_0][q_0, x, q_0] \\ [q_0, x, q_1] &\Rightarrow O[q_0, x, q_0][q_0, x, q_1] \\ [q_0, x, q_0] &\Rightarrow O[q_0, x, q_1][q_1, x, q_0] \\ [q_0, x, q_1] &\Rightarrow O[q_0, x, q_1][q_1, x, q_1] \end{aligned}$$

(29)

$$\bullet \delta(q_0, 1, x) = (q_1, \lambda)$$

$$[q_0, x, q_1] \Rightarrow 1$$

$$\bullet \delta(q_1, 1, x) = (q_1, \lambda)$$

$$[q_1, x, q_1] \Rightarrow 1$$

$$\bullet \delta(q_1, \lambda, z_0) = (q_1, \lambda)$$

$$[q_1, z_0, q_1] \Rightarrow \lambda$$

$$\begin{aligned} P: S &\Rightarrow A_3/A_4 \\ A_3 &\Rightarrow \emptyset A_1 A_3 / \emptyset A_2 A_7 \\ A_4 &\Rightarrow \emptyset A_1 A_4 / \emptyset A_2 A_8 \\ A_1 &\Rightarrow \emptyset A_1 A_1 / \emptyset A_2 A_5 \\ A_2 &\Rightarrow \emptyset A_1 A_2 / \emptyset A_2 A_6 / 1 \\ A_6 &\Rightarrow 1 \\ A_8 &\Rightarrow \lambda \end{aligned}$$

A_1, A_3, A_5, A_7 gemeinsame Ableitungen

$$P: S \Rightarrow A_4$$

$$A_4 \Rightarrow \emptyset A_2 A_8$$

$$A_2 \Rightarrow \emptyset A_2 A_6 / 1$$

$$A_6 \Rightarrow 1$$

$$A_8 \Rightarrow \lambda$$

$\rightarrow$ Bsp kann so abgeleitet werden $\rightarrow$

$$G = (V_N, V_T, P, S)$$

$$V_T = \{0, 1\} \quad V_N = \{S, A\}$$

$$P: S \Rightarrow \emptyset A_2$$

$$A \Rightarrow \emptyset A_1 / 1$$

